ZERO-TRUST MICRO-SEGMENTATION FIREWALL PROXY

An Identity-Aware Layer-7 Zero-Trust Proxy for HTTP Microservices

A PROJECT REPORT

Submitted by

Polisetti Hari Sri Raj Karthik (23BCY10308)

Sahil Shankar Thakkar (23BCY10043)

Shaivyaa Sharma (23BCY10020)

Subeer Srivastava (23BCY10143)

Sameep Upadhyay (23BCY10187)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

(Cyber Security and Digital Forensics)



SCHOOL OF COMPUTING SCIENCE AND ENGINEERING

VIT BHOPAL UNIVERSITY

KOTHRIKALAN, SEHORE

MADHYA PRADESH - 466114

OCTOBER 2026

VIT BHOPAL UNIVERSITY, KOTHRIKALAN, SEHORE MADHYA PRADESH – 466114

BONAFIDE CERTIFICATE

Certified that this project report titled **"Zero-Trust Micro-Segmentation Firewall Proxy — An Identity-Aware Layer-7 Zero-Trust Proxy for HTTP Microservices"** is the bonafide work of **Polisetti Hari Sri Raj Karthik (23BCY10308), Sahil Shankar Thakkar (23BCY10043), Shaivyaa Sharma (23BCY10020), Subeer Srivastava (23BCY10143), and Sameep Upadhyay (23BCY10187)** who carried out the project work under my supervision. Certified further that, to the best of my knowledge, the work reported at this time does not form part of any other project/research work based on which a degree or award was conferred on an earlier occasion on this or any other candidate.

PROGRAM CHAIR,
Dr. Ajay Kumar Phulre, Professor Sr.,
School of Computing Science and Engineering,
VIT Bhopal University.

PROJECT SUPERVISOR,
Dr. Sajjad Ahmed, Professor Sr.,
School of Computing Science and Engineering,
VIT Bhopal University.

The Capstone Project Examination is held on September-October 2026

ACKNOWLEDGEMENT

We would like to thank the Lord Almighty for His presence and immense blessings throughout the project work.

We wish to express our heartfelt gratitude to Dr. Ajay Kumar Phulre, Program Chair, Cyber Security and Digital Forensics, for his valuable support and encouragement in carrying out this work.

We would like to thank our supervisor, **Dr. Sajjad Ahmed**, for continually guiding and actively participating in our project — in particular for his architectural feedback after the first review, which directly shaped the consolidation of our design into the efficient gateway architecture presented in this report.

We would like to thank all the technical and teaching staff of the School of Computing Science and Engineering, who extended their support directly or indirectly.

Finally, we are deeply indebted to our parents, who have been our greatest support as we worked to make this project a success.

LIST OF ABBREVIATIONS

Abbreviation	Definition
API	Application Programming Interface
CA	Certificate Authority
CI	Continuous Integration
CISA	Cybersecurity and Infrastructure Security Agency
CNCF	Cloud Native Computing Foundation
CSR	Certificate Signing Request
DoS	Denial of Service
DNS	Domain Name System
HTTP	HyperText Transfer Protocol
JSONL	JavaScript Object Notation Lines (one JSON object per line)
L3/L4	Network / Transport layer (of the OSI model)
L7	Application layer (of the OSI model)
mTLS	Mutual Transport Layer Security
NIST	National Institute of Standards and Technology
OWASP	Open Web Application Security Project
P50/P99	50th / 99th percentile (latency measurement)
PBAC	Policy-Based Access Control
PKI	Public Key Infrastructure
REST	Representational State Transfer
RSA	Rivest–Shamir–Adleman (cryptosystem)
SAN	Subject Alternative Name (X.509 certificate extension)
SHA-256	Secure Hash Algorithm, 256-bit
SPIFFE	Secure Production Identity Framework For Everyone
SVID	SPIFFE Verifiable Identity Document
TLS	Transport Layer Security
VM	Virtual Machine
X.509	Standard defining public-key certificate format

Abbreviation	Definition
YAML	YAML Ain't Markup Language (human-readable config format)
ZTA	Zero-Trust Architecture
ZTNA	Zero-Trust Network Access

LIST OF FIGURES AND GRAPHS

FIGURE NO.	TITLE	PAGE NO.
1.1	Flat network (perimeter trust) versus zero-trust enforcement	1
1.2	Three-version design evolution timeline (V0.1 → V1.0 → V2.0)	3
2.1	Istio sidecar-mesh deployment model (illustrative comparison)	9
3.1	ZetaShield system context diagram	13
4.1	V2.0 consolidated two-hop gateway architecture	21
4.2	Request lifecycle sequence diagram (three-checkpoint pipeline)	22
4.3	Docker Compose network topology (zeta-net bridge isolation)	24
5.1	X.509 certificate showing SPIFFE URI in the SAN extension	28
5.2	Seven-test adversarial suite — terminal output (client)	33
5.3	Gateway enforcement log with SPIFFE identities (server)	34
5.4	Docker Compose port mapping — backends with no host ports	30
5.5	Latency benchmark output (direct vs. gateway, P50/P99)	35
5.6	GitHub Actions CI run — 19 tests passing	32
5.7	Structured audit log (audit.log.jsonl)	34
5.8	Policy hot-reload — gateway detecting policy.yaml change	28
6.1	Real-world deployment scenario (fintech internal services)	40

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Comparative analysis of existing zero-trust enforcement systems	10
3.1	Hardware and software requirements	13
3.2	Functional requirements	15
4.1	Threat model — six adversary scenarios and countermeasures	23
4.2	policy.yaml declarative rule schema	23
5.1	Seven-test adversarial verification matrix	32
5.2	Unit test category coverage (19 tests)	31
5.3	Latency benchmark results (500 requests)	35
6.1	Mapping of review feedback to implemented features	38
6.2	Real-world applicability scenarios	39
7.1	Constraints and planned Phase 2 enhancements	41

ABSTRACT

PURPOSE: Modern microservice applications communicate over internal HTTP networks that are structurally flat: once perimeter defences are breached, an attacker moves laterally between services with no further access checks. Broken Access Control is ranked the number one web application security threat (OWASP A01:2025), and NIST SP 800-207A and CISA guidance identify micro-segmentation with cryptographic workload identity as the mandated countermeasure. This project addresses that gap by designing and implementing ZetaShield, an identity-aware Layer-7 zero-trust gateway for HTTP microservices.

METHODOLOGY: The system was developed through three iterative versions informed by formal review feedback: V0.1, a four-hop sidecar proxy prototype with header-based identity; V1.0, a consolidated two-hop gateway enforcing mutual TLS (mTLS) with service identities extracted from X.509 Subject Alternative Name fields in SPIFFE URI format, a declarative deny-by-default policy engine (policy.yaml), and structured append-only JSONL audit logging; and V2.0, which containerizes the entire mesh using Docker Compose with network-namespace isolation, adds zero-downtime policy hot-reload, and extends the demonstration suite to seven adversarial tests. The gateway is written in Go using only the standard library plus a YAML parser; backends are dependency-free Python services. Verification comprises 19 table-driven unit tests executed in a GitHub Actions continuous integration pipeline on every commit, a seven-test adversarial demonstration suite covering authorized access, anonymous rejection, forged-

certificate rejection, deny-by-default enforcement, cross-service micro-segmentation, and fail-closed behaviour on backend outage, and a 500-request latency benchmark comparing direct and gateway-mediated requests.

FINDINGS: Full zero-trust enforcement — mTLS session establishment, cryptographic identity extraction, policy evaluation, routing, and audit logging — adds a median (P50) latency of approximately 0.6 milliseconds per request and under 1.2 milliseconds at the 99th percentile. Backends deployed with no published host ports are unreachable except through the gateway, making lateral movement physically impossible rather than merely policy-discouraged. The project demonstrates that the core enforcement layer of commercial service meshes can be built from scratch in a compact, fully testable, reproducible, zero-cost form, and is deployable with a single Docker Compose command.

TABLE OF CONTENTS

LIST OF ABBREVIATIONS ..... **iii**

LIST OF FIGURES AND GRAPHS ..... **v**

LIST OF TABLES ..... **vi**

ABSTRACT ..... **vii**

CHAPTER 1: PROJECT DESCRIPTION AND OUTLINE **1**

1.1 Introduction 1

1.2 Motivation for the Work 2

1.3 About the Project and Techniques Employed 3

1.5 Problem Statement 4

1.6 Objectives of the Work 4

1.7 Organization of the Report 5

1.8 Summary 6

CHAPTER 2: RELATED WORK INVESTIGATION **7**

2.1 Introduction 7

2.2 Zero-Trust Architecture and Micro-Segmentation (Core Area) 7

2.3 Existing Approaches / Methods 8

2.3.1 Production Service Meshes (Istio, Linkerd, HashiCorp Consul) 8

2.3.2 Workload Identity Frameworks (SPIFFE / SPIRE) 9

2.3.3 Kernel-Level and Overlay Approaches (Cilium, OpenZiti, Octelium) 9

2.3.4 External Authorization and Prior Academic Implementations 10

2.4 Pros and Cons of the Stated Approaches / Methods 10

2.5 Issues / Observations from Investigation 11

2.6 Summary 12

CHAPTER 3: REQUIREMENT ARTIFACTS **13**

3.1 Introduction 13

3.2 Hardware and Software Requirements 13

3.3 Specific Project Requirements 14

3.3.1 Data Requirements 14

3.3.2 Functional Requirements 15

3.3.3 Performance and Security Requirements 16
3.3.4 Look and Feel Requirements 17
3.3.5 Operational and Deployment Requirements 17
3.4 Summary 18
CHAPTER 4: DESIGN METHODOLOGY AND ITS NOVELTY 19
4.1 Methodology and Goal 19
4.2 Functional Modules Design and Analysis 20
4.3 Software Architectural Design 21
4.3.1 The Consolidated Two-Hop Gateway 21
4.3.2 The Three-Checkpoint Enforcement Pipeline 22
4.3.3 Threat Model and Countermeasure Mapping 23
4.3.4 Policy Schema Design 23
4.3.5 Containerized Network Isolation Design (V2.0) 24
4.3.6 Zero-Downtime Policy Hot-Reload Design 25
4.4 Subsystem Services 25
4.5 User Interface Designs 25
4.6 Novelty of the Design 26
4.7 Summary 26
CHAPTER 5: TECHNICAL IMPLEMENTATION AND ANALYSIS 27
5.1 Outline 27
5.2 Technical Coding and Code Solutions 27
5.2.1 Repository Structure 27
5.2.2 Key Implementation Solutions 27
5.2.3 PKI Integration and Resolved Interoperability Defects 29
5.3 Working Layout of Forms (Demonstration Environment) 30
5.4 Prototype Submission 30
5.5 Test and Validation 31
5.5.1 Unit Test Suite (19 Tests) 31
5.5.2 The Seven-Test Adversarial Suite 32
5.5.3 Validation Summary 34
5.6 Performance Analysis 34

5.6.1 Methodology 34
5.6.2 Results 35
5.6.3 Analysis 35
5.7 Summary 36
CHAPTER 6: PROJECT OUTCOME AND APPLICABILITY 37
6.1 Outline 37
6.2 Key Implementation Outlines of the System 37
6.3 Significant Project Outcomes 37
6.4 Project Applicability on Real-World Applications 39
6.5 Inference 40
CHAPTER 7: CONCLUSIONS AND RECOMMENDATION 41
7.1 Outline 41
7.2 Conclusions 41
7.3 Limitations / Constraints of the System 41
7.4 Future Enhancements (Phase 2 Roadmap) 42
7.5 Recommendation 43
REFERENCES 44
Appendix A — Reproduction Commands (Clean Machine) 46
Appendix B — Figure Sources (for your screenshot inserts) 47
Appendix C — Version History Summary 48

Note: This Table of Contents is auto-generated. After inserting figures or making edits, right-click it and choose "Update Field" → "Update entire table" to refresh the page numbers.

CHAPTER 1: PROJECT DESCRIPTION AND OUTLINE

1.1 Introduction

Modern enterprise and cloud-native applications are constructed as decoupled microservices that communicate internally over HTTP REST APIs. However, legacy security architectures concentrate enforcement at the network perimeter — a model that produces a "hard shell, soft interior" topology. Once the perimeter is breached through a phished credential, a compromised container, or a vulnerable dependency, the internal network is flat: every service can reach every other service with no further authentication or authorization checks. Threat actors and ransomware exploit exactly this property, moving laterally from an entry point to databases, administrative interfaces, and storage systems while perimeter defences observe nothing.

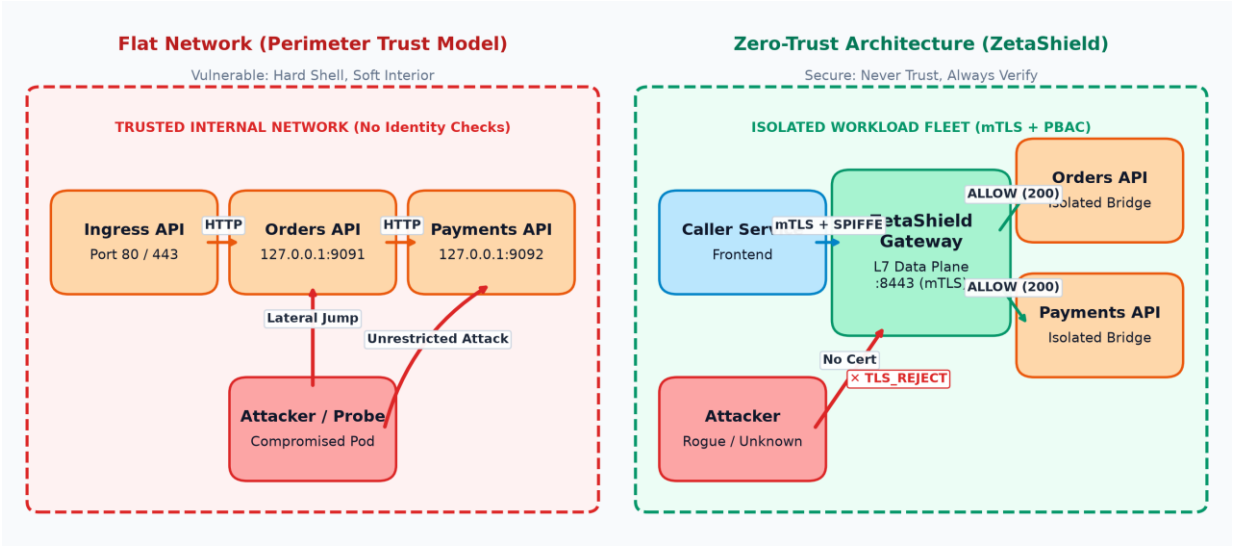


Figure 1.1: Flat network (perimeter trust) versus zero-trust enforcement

This report presents **ZetaShield**, a Zero-Trust Micro-Segmentation Firewall Proxy — an identity-aware, Layer-7 security gateway for HTTP microservices. The system enforces the principle of "never trust, always verify" at the level of every individual service-to-service call: each caller must present a cryptographic X.509 certificate, its identity is extracted from the certificate's Subject Alternative Name in SPIFFE URI format, every request is evaluated against a declarative deny-by-default policy, and every decision is written to a structured, append-only audit log. Unauthorized requests never reach the protected backend; failure of any component causes the system to fail closed rather than fail open.

The project is developed by Group 39 under course code DSN4091 (Capstone Project, Phase 1) under the supervision of Dr. Sajjad Ahmed, and is fully software-defined: it runs on commodity hardware, uses exclusively open-source tooling, and carries a zero-dollar external budget. The

complete source code, test suite, and continuous integration pipeline are publicly available at https://github.com/karthik5366/Zero-Trust-Micro-Proxy.

1.2 Motivation for the Work

Three observations motivated this project:

First, the failure mode of perimeter security is well documented and intensifying. The Open Web Application Security Project (OWASP) ranks Broken Access Control as the number one web application security risk in its 2025 edition (A01:2025). The National Institute of Standards and Technology (NIST), in Special Publication 800-207A, identifies sidecar proxies and cryptographic workload identity as foundational building blocks of a Zero-Trust Architecture. The Cybersecurity and Infrastructure Security Agency (CISA), in its 2025 guidance, designates micro-segmentation as an essential countermeasure against lateral movement. These are not hypothetical concerns: breaches in finance, healthcare, and government systems routinely escalate from a single compromised credential to infrastructure-wide compromise precisely because internal networks grant implicit trust.

Second, the industry's east-west traffic problem is growing. The Cloud Native Computing Foundation (CNCF) annual survey reports that over 80% of container users run Kubernetes in production, meaning service-to-service ("east-west") traffic now dwarfs traditional north-south traffic. Yet the security tooling that protects east-west communication — service meshes such as Istio — is heavyweight, complex to operate, and opaque: hundreds of thousands of lines of code standing between an engineering team and an understanding of their own security enforcement.

Third, and most directly, a formal academic review of the initial prototype (V0.1) identified three concrete engineering deficiencies: excessive architectural layering introducing latency, a single protected service that did not demonstrate true micro-segmentation, and a deployment footprint limited to a single developer machine. Each of these criticisms was deliberately converted into a design objective for the current version, and each is answered by a measurable, testable feature described in this report.

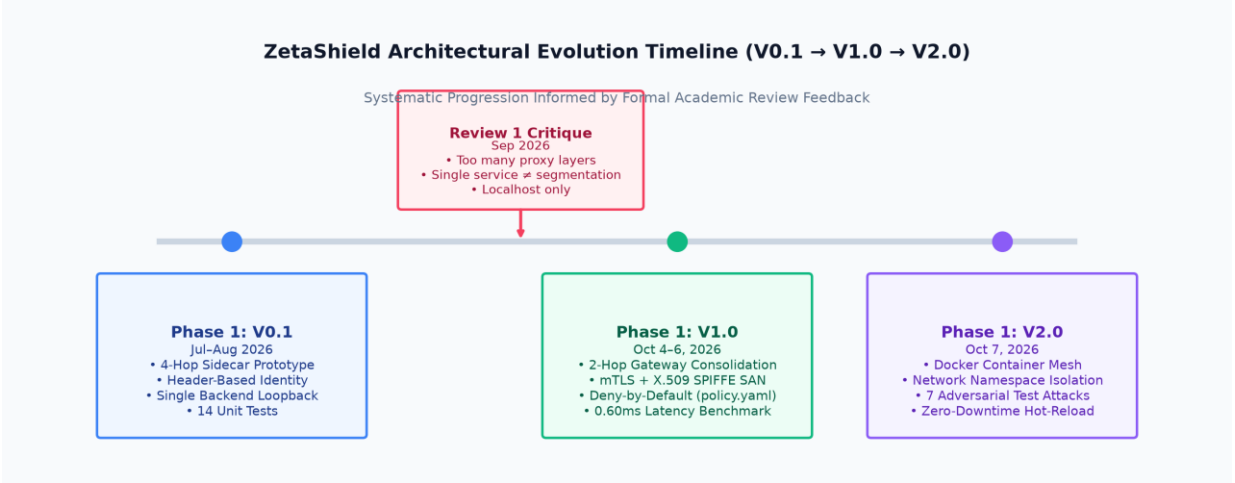


Figure 1.2: Three-version design evolution timeline (V0.1 → V1.0 → V2.0)

1.3 About the Project and Techniques Employed

ZetaShield is a consolidated Layer-7 reverse proxy that sits in the request path between calling services and protected backend services. Its enforcement pipeline applies three mandatory checkpoints to every request:

1. **Identity (mTLS):** The gateway terminates mutual TLS, requiring and cryptographically verifying a client certificate chained to a private Certificate Authority. The caller's identity — a SPIFFE-format URI such as `spiffe://zetashield.local/ns/default/sa/frontend-service` — is extracted from the verified certificate's Subject Alternative Name. Callers without a trusted certificate are rejected at the TLS handshake and never reach the application layer.
2. **Authorization (deny-by-default policy):** The caller identity, request path, and HTTP method are matched against a declarative `policy.yaml` ruleset. Only an explicit match permits forwarding; every unmatched request receives HTTP 403 and is logged. An empty or corrupted policy file therefore produces total lockdown — the safe state is the default state.
3. **Routing and audit:** Permitted requests are routed by longest-prefix matching to the appropriate isolated backend, and every decision (ALLOW, DENY, or ERROR) is synchronously appended to a structured JSONL audit log with timestamp, identity, method, path, and reason.

The principal techniques employed are: **Zero-Trust Architecture** (NIST SP 800-207 alignment); **mutual TLS with X.509 PKI**, including a locally operated Root Certificate Authority and per-service certificates with SAN identity extensions; **SPIFFE-style workload identity encoding** (the

same scheme used by production ecosystems such as Istio and SPIRE); **declarative policy-based access control** structurally equivalent to Kubernetes NetworkPolicy and Istio AuthorizationPolicy; **fail-closed error handling**; **containerized network-namespace isolation** using Docker Compose, in which backends publish no host ports and are reachable only through the gateway; **zero-downtime policy hot-reload**; and **continuous integration** with automated vetting, building, and unit testing on GitHub's infrastructure.

The implementation language for the gateway is **Go**, chosen for its per-connection concurrency model (goroutines), its production-grade standard-library reverse proxy (`net/http/httputil`) and TLS stack, and its compilation to a single dependency-free static binary. Protected backends are written in dependency-free Python to eliminate installation risk. The system's only external library dependency is a YAML parser for policy files.

1.5 Problem Statement

*Traditional firewall architectures enforce security exclusively at Layers 3 and 4 on the network border. In microservice environments this produces three structural flaws: (1) **implicit internal trust** — any workload inside the network enjoys unrestricted communication with every other workload; (2) **unreliable IP-based identity** — in containerized environments, IP addresses are ephemeral and routinely recycled, rendering IP-based access rules both spoofable and semantically meaningless; and (3) **coarse-grained visibility** — Layer 3/4 devices inspect addresses and ports, not HTTP methods or API paths, leaving internal APIs unmonitored and uncontrolled. The consequence is unconstrained lateral movement following any perimeter breach.*

*The problem addressed by this project is therefore: **can a software-defined, Layer-7 gateway enforce cryptographic, identity-aware, deny-by-default authorization on every east-west HTTP call between microservices — with deterministic fail-closed behaviour, complete auditability, container-native deployment, and sub-millisecond latency overhead — without dependency on a commercial service mesh, a cloud subscription, or specialized hardware?***

1.6 Objectives of the Work

1. To design and implement a Layer-7 reverse proxy that intercepts all HTTP traffic between microservices and applies identity verification, policy authorization, routing, and audit logging on every request.
2. To establish a local Public Key Infrastructure and enforce mutual TLS, extracting unforgeable service identity from X.509 Subject Alternative Name fields in SPIFFE URI format.

3. To implement a declarative, deny-by-default policy model (identity + path + HTTP method) with an implicit safe state, structurally equivalent to industry-standard policy primitives.
4. To demonstrate true micro-segmentation across a multi-backend service fleet, in which authenticated services are cryptographically prevented from accessing one another without explicit permission.
5. To enforce deterministic fail-closed behaviour for every failure mode, including missing certificates, forged certificates, policy gaps, and backend outages.
6. To containerize the complete system such that the entire mesh — gateway and isolated backends — deploys with a single Docker Compose command, with backends unreachable except through the gateway.
7. To quantify the performance cost of zero-trust enforcement through repeatable latency benchmarking.
8. To verify the system through automated unit testing (19 tests) in a continuous integration pipeline, and through a seven-test adversarial demonstration suite covering every security guarantee.

1.7 Organization of the Report

The remainder of this report is organized as follows. **Chapter 2** investigates related work: production service meshes (Istio, Linkerd, Consul), identity frameworks (SPIFFE/SPIRE), kernel-level enforcement (Cilium), zero-trust overlays (OpenZiti, Octelium), and prior academic implementations, identifying the gap this project fills. **Chapter 3** specifies the requirement artifacts — hardware and software requirements, and the data, functional, performance, and security requirements. **Chapter 4** presents the design methodology and its novelty: the consolidated two-hop architecture, the three-checkpoint enforcement pipeline, the threat model, and the container isolation design. **Chapter 5** details the technical implementation and analysis: code structure, the PKI and mTLS integration, the policy engine and hot-reload mechanism, Docker deployment, the test and validation strategy, and the latency benchmark results. **Chapter 6** presents the project outcome and its applicability to real-world scenarios, including the mapping of review feedback to delivered features. **Chapter 7** concludes with limitations, and recommended future enhancements for Phase 2. References and appendices follow.

1.8 Summary

This chapter introduced ZetaShield, an identity-aware Layer-7 zero-trust gateway that eliminates implicit internal network trust by cryptographically authenticating every calling service, authorizing every HTTP request against a deny-by-default policy, isolating backends at the network layer, and auditing every decision. The motivation — spanning industry standards, the growth of east-west traffic, and formal review feedback — was presented, the problem was stated precisely, and eight measurable objectives were defined. The subsequent chapters develop the related-work context, requirements, design, implementation, and evaluation of the system that fulfils these objectives.

CHAPTER 2: RELATED WORK INVESTIGATION

2.1 Introduction

A rigorous related-work investigation serves two purposes in this project: first, to establish that the problem being solved — lateral movement across flat internal networks — is formally recognized by standards bodies and actively engineered against by the industry; and second, to identify precisely which capabilities already exist, in what form, and at what cost, so that the contribution of the present work can be stated honestly and precisely. The investigation surveyed four categories of existing work: (i) the standards and academic literature defining Zero-Trust Architecture and micro-segmentation; (ii) production service meshes that implement identity and policy enforcement at scale; (iii) workload-identity frameworks; and (iv) kernel-level, overlay-network, and external-authorization approaches, together with prior academic implementations. The survey was conducted against public documentation, peer-reviewed literature, and openly available source repositories during July–August 2026.

2.2 Zero-Trust Architecture and Micro-Segmentation (Core Area)

The foundational reference for this project's core area is the work of Rose et al. [1], whose NIST Special Publication 800-207 provides the abstract definition of Zero-Trust Architecture (ZTA). The publication establishes that zero trust is an architectural philosophy rather than a product: no implicit trust is granted to any user, device, or workload based on network location or asset ownership, and every access request must be authenticated, authorized, and continuously validated. Its successor, NIST SP 800-207A [2], moves from definition to construction and identifies the specific building blocks relevant to this project: sidecar proxies, API gateways, and cryptographic workload identity as the enforcement mechanisms of a zero-trust deployment.

Regulatory convergence reinforces the academic framing. The Open Web Application Security Project ranks Broken Access Control as the single most critical web application security risk in its 2025 edition (A01:2025) [3] — a direct consequence of flat-network trust assumptions. The Cybersecurity and Infrastructure Security Agency, through its Zero Trust Maturity Model and 2025 micro-segmentation guidance [4], designates workload-level segmentation as an essential countermeasure against lateral movement. Industry adoption data contextualizes the urgency: the Cloud Native Computing Foundation's annual survey [5] reports that approximately 82% of container users run Kubernetes in production, meaning that east-west (service-to-service) traffic has become the dominant traffic class in modern deployments — and simultaneously the least authenticated.

Academic analysis of zero-trust adoption reveals both momentum and immaturity. Yeoh et al. [6] identify critical success factors for organizational zero-trust adoption and propose a staged implementation approach, observing that most deployments begin with identity and access management and only later address workload-level segmentation. A survey by Tornes et al. [7] on the theory and application of zero-trust security concludes that despite substantial industrial investment, academic research on concrete zero-trust implementations "is still in its infancy," with most published work addressing policy or architecture rather than reproducible enforcement systems. A 2025 master's study at Delft University of Technology [8] examines micro-segmentation applied in legacy-heavy enterprise environments and documents the practical trade-offs — operational complexity, policy sprawl, and tooling opacity — that obstruct adoption in organizations without dedicated platform-engineering teams. These observations directly inform the present project's motivation: a compact, transparent, reproducible enforcement layer has documented value beyond the classroom.

2.3 Existing Approaches / Methods

2.3.1 Production Service Meshes (Istio, Linkerd, HashiCorp Consul)

The dominant industry answer to east-west security is the service mesh. **Istio** [9] implements the architecture this project originally prototyped in its V0.1 form: an injected sidecar proxy (Envoy) per workload, a centralized control plane (istiod) distributing certificates and configuration, automatic mutual TLS between meshed workloads, and L7 authorization through declarative AuthorizationPolicy resources supporting ALLOW, DENY, and CUSTOM actions. Notably, the Istio project's own development trajectory validates a central design decision of the present work: the introduction of **ambient mode** — a sidecar-less deployment — was driven precisely by the resource overhead, latency cost, and operational complexity of per-workload sidecars, the same "too many layers" critique raised during this project's first review.

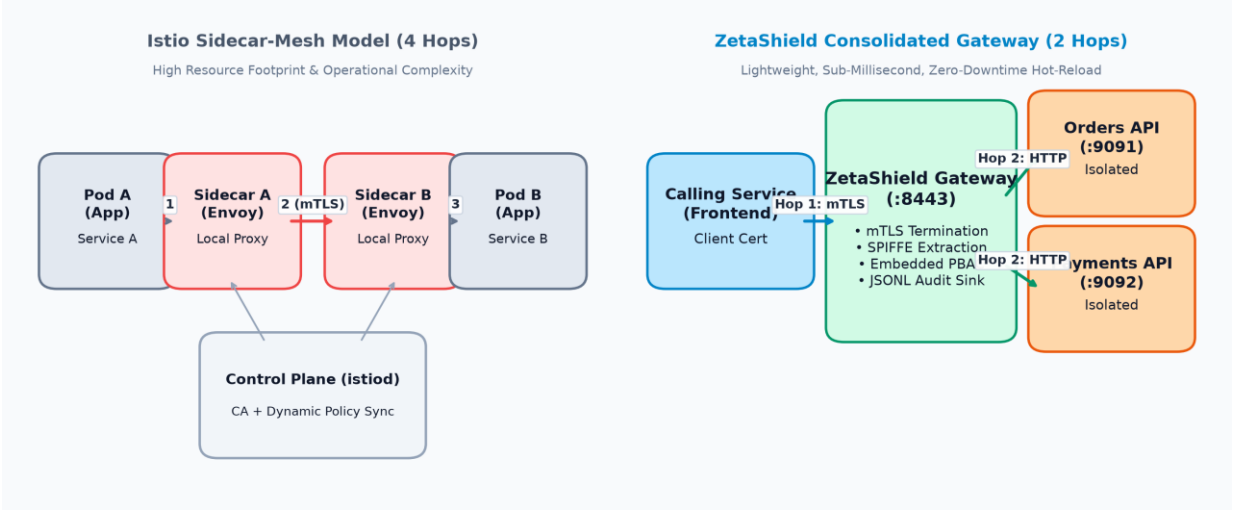


Figure 2.1: Istio sidecar-mesh deployment model (illustrative comparison)

Linkerd [10] takes a lighter approach: a purpose-built Rust micro-proxy, automatic mTLS for all TCP traffic between meshed pods, and identity managed by the control plane. Linkerd demonstrates that a mesh can be materially simpler than Istio while retaining transport-level authentication, though its L7 authorization granularity is coarser. **HashiCorp Consul** [11] generalizes the mesh beyond Kubernetes with "intentions" — service-to-service access rules evaluated at L4 for TCP traffic and L7 for HTTP, enforced through Connect sidecars — at the cost of operating a Consul cluster.

2.3.2 Workload Identity Frameworks (SPIFFE / SPIRE)

The **SPIFFE** (Secure Production Identity Framework For Everyone) specification [12], and its reference implementation **SPIRE**, constitute the industry standard for workload identity — the "who are you?" layer of zero trust. SPIFFE defines a URI-based identity scheme (e.g., `spiffe://trust-domain/path/service`) carried in X.509 certificate Subject Alternative Name extensions (X.509-SVIDs), with short validity windows (typically one hour) rotated automatically by node-local agents. SPIRE is a graduated CNCF project and is integrated into production ecosystems including Istio and Envoy. Critically for the present investigation, SPIFFE/SPIRE is deliberately an *identity-only* framework: it issues and verifies credentials but enforces no access policy and proxies no traffic. The present project adopts the SPIFFE identity encoding (Section 4.3) precisely for this production compatibility.

2.3.3 Kernel-Level and Overlay Approaches (Cilium, OpenZiti, Octelium)

Cilium [13] enforces L7 policy in the Linux kernel via eBPF, with an embedded Envoy proxy performing HTTP-level authorization on each node. It achieves remarkable efficiency and scale

but is bound to Linux kernel capabilities — a portability constraint rather than a universal mechanism. **OpenZiti** [14] inverts conventional exposure: an overlay network in which services are "dark" — invisible and unreachable — until a cryptographic identity is authorized by a central controller; connectivity is established through tunnelers or an SDK. **Octelium** [15] extends this pattern into a unified zero-trust platform spanning remote access, ZTNA, and API gateway functions. These systems demonstrate the "deny-by-default at the network layer" philosophy the present project implements in miniature, but they are full network platforms rather than focused, inspectable enforcement engines.

2.3.4 External Authorization and Prior Academic Implementations

A complementary industry pattern decouples enforcement from decisioning: **Envoy's external authorization filter (ext_authz)** [16] pauses each request and consults an external policy engine — commonly the Open Policy Agent (OPA) [17] — before forwarding. This separation of data plane and policy decision point mirrors the design of the present project's policy architecture, though ZetaShield embeds the evaluation in-process for latency reasons (Chapter 5). Finally, prior academic and capstone implementations were surveyed: typical examples include enterprise zero-trust network designs for healthcare [18] constructed in Packet Tracer and cloud consoles, and SDN-based zero-trust provisioning projects [19]. A consistent pattern emerged across these works: they *configure* existing commercial or open-source tooling to demonstrate zero-trust principles, rather than *constructing* the enforcement mechanism itself — a distinction that defines the contribution of the present project.

2.4 Pros and Cons of the Stated Approaches / Methods

Table 2.1: Comparative analysis of existing zero-trust enforcement systems

System / Work	Identity Mechanism	L7 Policy	Deployment Requirement	Principal Limitation Relative to This Project
Istio [9]	SPIFFE-style X.509 SVIDs, auto-issued	AuthorizationPolicy (ALLOW/DENY/CUSTOM)	Kubernetes + istiod control plane	Hundreds of thousands of LOC; deployment is configuration, not construction; heavy operational surface

System / Work	Identity Mechanism	L7 Policy	Deployment Requirement	Principal Limitation Relative to This Project
Linkerd [10]	Automatic mTLS, control-plane-issued	Coarser L7 granularity	Kubernetes	Transport-focused; K8s-bound
Consul [11]	mTLS via Connect CAs	Intentions (L4/L7)	Consul cluster + sidecars	Distributed-systems operational burden
SPIFFE/SPIRE [12]	X.509-SVID standard, 1-h TTL rotation	None (identity only)	SPIRE server + node agents	No enforcement plane; must be composed with a proxy
Cilium [13]	K8s label identity	L7 via node-local Envoy	Linux kernel / eBPF	Kernel-bound; not userspace-portable
OpenZiti / Octelium [14][15]	Cryptographic overlay identity	Controller-authorized services	Full overlay platform	Platform scope exceeds focused L7 enforcement
Envoy ext_authz + OPA [16][17]	Delegated to external PDP	External policy evaluation	Envoy + OPA deployment	Composition required; enforcement engine itself not built
Prior academic implementations [18][19]	Vendor-dependent	Vendor-dependent	Varies	Configure existing tools; enforcement mechanism not constructed

The production meshes solve identity and policy at scale but obscure the mechanics behind massive codebases and mandatory infrastructure. The identity frameworks solve identity alone. The kernel and overlay systems solve isolation but at platform scope. The academic works demonstrate principles through configuration of third-party products.

2.5 Issues / Observations from Investigation

Five observations emerge from this investigation, each shaping a design decision of the present project:

1. **Every production system conflates scale requirements with mechanism.** The core enforcement loop — verify certificate, extract identity, evaluate policy, route, log — is conceptually small, yet no surveyed system exposes it in a form a student team can read end-to-end, modify, and defend line-by-line.
2. **The industry itself is retreating from architectural complexity.** Istio's ambient mode exists because sidecars cost latency and memory [9] — empirical validation of the consolidation feedback this project received at its first review, and the direct justification for ZetaShield's two-hop gateway design.
3. **The SPIFFE identity encoding is the de-facto standard** [12] and is directly adoptable in a from-scratch implementation, giving student-grade work production-grade identity semantics.
4. **Deny-by-default must be structural, not configured.** Across surveyed systems, safety derives from the absence of allowance rather than the presence of denial — a property ZetaShield adopts as a design invariant.
5. **No existing open-source project combines** a from-scratch L7 proxy, X.509 SAN/SPIFFE identity extraction, an embedded deny-by-default policy engine, fail-closed semantics, container-native isolation, and reproducible benchmarking in a single dependency-light, laptop-runnable repository. This is the gap the present work fills: a complete, readable reference implementation of the NIST SP 800-207A enforcement pattern [2].

2.6 Summary

This chapter surveyed the standards landscape (NIST, CISA, OWASP, CNCF), the academic literature on zero-trust maturity and micro-segmentation practice, and four categories of existing implementations: production service meshes, workload-identity frameworks, kernel/overlay systems, and prior academic configurations. The investigation established both the urgency of the problem and a precise gap: the absence of a compact, from-scratch, fully tested and benchmarked enforcement engine implementing the documented zero-trust pattern. Chapter 3 translates this problem context and gap into concrete requirement artifacts for the system.

CHAPTER 3: REQUIREMENT ARTIFACTS

3.1 Introduction

The requirements for ZetaShield were derived from three sources: the problem statement and threat model (Section 1.5), the industry standards investigated in Chapter 2, and the formal review feedback received on the V0.1 prototype — which explicitly demanded reduced architectural layering, genuine multi-service micro-segmentation, containerized deployability, and quantified efficiency. Requirements were specified before the V1.0 and V2.0 implementation cycles and are stated here in verifiable form; Chapter 5 maps each requirement to its implementation and test evidence. The overarching constraint governing all requirements is the project's zero-budget, fully software-defined charter: the complete system must build, test, and deploy on commodity hardware with exclusively open-source tooling.

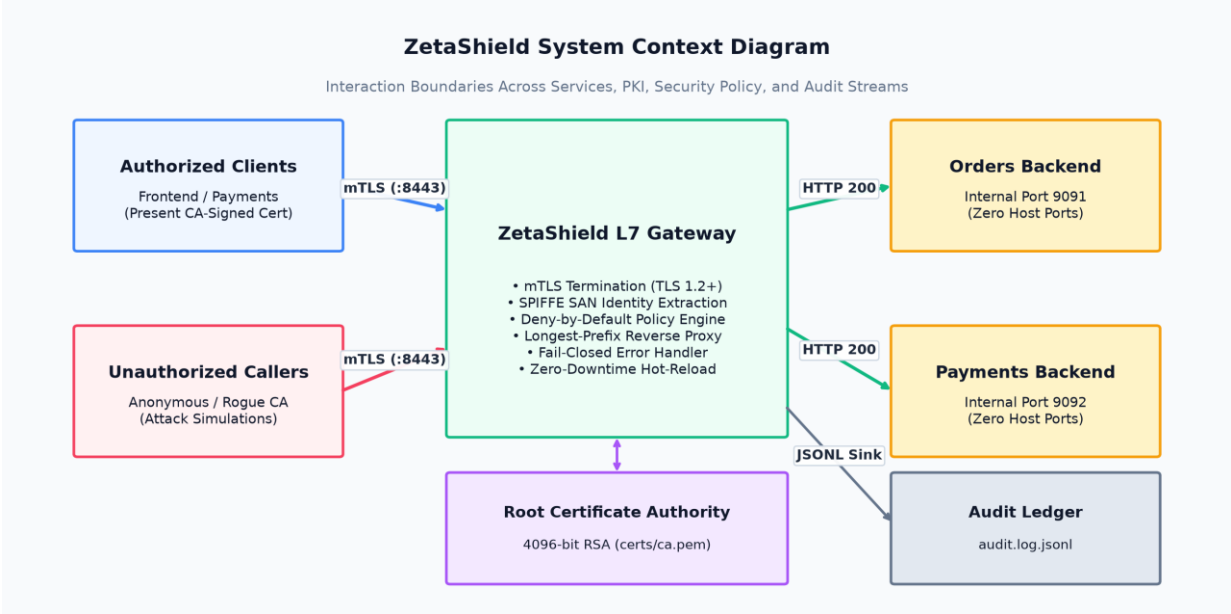


Figure 3.1: ZetaShield system context diagram

3.2 Hardware and Software Requirements

Hardware (development and demonstration machine):

Table 3.1: Hardware and software requirements

Component	Specification
Processor	AMD Ryzen 5 5600H — 6 cores / 12 threads
Memory	16 GB DDR4 (full Docker mesh peak utilization < 3 GB)

Component	Specification
Storage	< 500 MB project footprint including container images
Network	Loopback and Docker bridge networking only; no external services required
Specialized hardware	None — 100% software-defined

Software:

Component	Version / Nature	Role
Go (Golang)	1.21+	Gateway implementation language
gopkg.in/yaml.v3	v3.0.1 (sole external dependency)	Policy file parsing
Python	3.8+, standard library only	Protected backend services and adversarial test client
OpenSSL	3.x	Root CA generation, certificate issuance, adversarial asset generation
Docker Desktop / Docker Compose	v2 compose syntax	Containerized deployment and network isolation
Git + GitHub	—	Version control; branch/PR/merge workflow; tagged releases
GitHub Actions	ubuntu-latest runners	Continuous integration (vet, build, 19 unit tests per push)
Operating system	Windows 10/11 (dev), Linux (CI), Alpine Linux (containers)	Cross-platform by design

Budget constraint: zero external expenditure — no paid cloud subscriptions, licenses, or hardware. The complete system is reproducible on any developer laptop meeting the above software requirements.

3.3 Specific Project Requirements

3.3.1 Data Requirements

The system manipulates four categories of data, each with a defined schema.

(a) Policy data (`policy.yaml`). A declarative YAML document constituting the single source of truth for routing and authorization. The schema comprises a routes map (path prefix → backend URL, resolved by longest-prefix match) and an ordered rules list, each rule specifying

`identity` (SPIFFE URI), `path_prefix`, and permitted `methods`. The absence of a matching rule is semantically a denial; an empty file therefore defines total lockdown.

(b) Identity data (X.509 certificates). Service identity is carried in the Subject Alternative Name URI field of CA-signed certificates: `URI:spiffe://zetashield.local/ns/default/sa/<service>`. The Root CA is a 4096-bit RSA self-signed certificate (10-year validity) with critical `keyUsage: keyCertSign` and `basicConstraints: CA:TRUE` extensions; service certificates are 2048-bit RSA, 30-day validity. The gateway's own certificate additionally embeds `IP:127.0.0.1` and `DNS:localhost` SAN entries for local TLS hostname verification.

(c) Audit data (`audit.log.jsonl`). Every enforcement decision is appended as one self-contained JSON object: `{timestamp (UTC, RFC 3339), decision (ALLOW/DENY/ERROR), identity, method, path, reason}`. The JSON Lines format was selected for its O(1) append semantics, crash safety (a failed write costs at most one record), and streamability for future observability tooling.

(d) Configuration data (environment variables). All filesystem paths and the listen address are configurable via `ZS_POLICY_PATH`, `ZS_CA_PATH`, `ZS_CERT_PATH`, `ZS_KEY_PATH`, `ZS_AUDIT_PATH`, and `ZS_LISTEN_ADDR`, with local-mode defaults; backend services accept `BIND_HOST`, `PORT`, and `APP_FILE`. This enables identical binaries in local and containerized operation.

3.3.2 Functional Requirements

Table 3.2: Functional requirements

ID	Requirement
FR-01	The gateway shall intercept every HTTP request between calling services and protected backends; no request shall reach a backend except through the gateway's enforcement pipeline.
FR-02	The gateway shall require and cryptographically verify a client X.509 certificate chained to the system's Root CA on every connection (`RequireAndVerifyClientCert`); connections failing verification shall be terminated at the TLS handshake.
FR-03	The gateway shall extract the caller's identity from the verified certificate's SAN URI field; no header-based or IP-based identity shall be accepted.
FR-04	The gateway shall evaluate every request against the declarative policy across the match dimensions identity, path prefix, and HTTP method, and shall deny-by-default any request not explicitly allowed.

ID	Requirement
FR-05	The gateway shall route permitted requests to backends by longest-prefix matching of the request path.
FR-06	Upon backend unreachability, the gateway shall return HTTP 503 and log the failure (fail-closed); no fallback path shall bypass enforcement.
FR-07	The gateway shall synchronously append a structured audit record for every decision, including denials and errors, with the decision reason.
FR-08	The gateway shall detect modifications to the policy file and reload rules without process restart or dropped connections (hot-reload), with reload events logged.
FR-09	The complete system — gateway and backends — shall deploy via a single `docker compose up` command, with backends publishing no host ports and reachable only through the gateway on the internal bridge network.
FR-10	All operational parameters shall be configurable via environment variables, preserving identical behavior in local and containerized modes.
FR-11	The system shall include an adversarial demonstration client executing a deterministic seven-test sequence covering authorized access, anonymous rejection, forged-certificate rejection, deny-by-default, micro-segmentation, own-domain access, and fail-closed behaviour.
FR-12	Every push to the repository shall trigger automated vetting, building, and unit testing (19 tests) on independent CI infrastructure.

3.3.3 Performance and Security Requirements

Performance requirements:

ID	Requirement	Achieved (Ch. 5)
PR-01	Median (P50) enforcement overhead — mTLS, identity extraction, policy evaluation, routing, audit — shall not exceed 1 ms per request on commodity hardware	~0.60 ms
PR-02	P99 enforcement overhead shall not exceed 2 ms	~1.18 ms
PR-03	Policy hot-reload shall take effect within 2 seconds of file modification	≤ 1 s
PR-04	The system shall handle concurrent connections without shared-state corruption (mutex-guarded policy and audit state)	Verified by design

Security requirements: Each adversarial test in the seven-test suite (Section 5.5) *is* a security requirement, stated as a guarantee:

ID	Guarantee
SR-01	No valid CA-signed certificate → connection rejected at TLS handshake (no application-layer exposure)
SR-02	Forged certificate (valid identity claim, untrusted issuer) → rejected at TLS handshake
SR-03	Valid identity, no explicit ALLOW rule → HTTP 403; the request never reaches routing or the backend
SR-04	Authenticated service accessing another service's domain without permission → HTTP 403 (micro-segmentation)
SR-05	Backend outage → HTTP 503 fail-closed; infrastructure failure never converts into permission
SR-06	Backends shall be network-unreachable except via the gateway — by loopback binding (local mode) or unpublished ports on an isolated bridge network (container mode)
SR-07	No private key material shall ever enter the version-control repository (scripted generation, `.gitignore` enforcement)

3.3.4 Look and Feel Requirements

The system's user-facing surfaces are operational rather than graphical in Phase 1: (i) the gateway's console output shall present each enforcement decision in a single scannable line with color-coded decision, caller identity, method, path, and reason; (ii) `policy.yaml` shall remain human-readable and self-documenting, usable as the primary administrative interface; (iii) the repository README shall serve as the deployment interface, enabling reproduction of the full demonstration on a clean machine via copy-paste commands; and (iv) the adversarial client's output shall state each test's scenario and result unambiguously, including the expected sequence for verification. A graphical administrative console is specified as a Phase 2 deliverable (Chapter 7).

3.3.5 Operational and Deployment Requirements

The system shall be deployable in two modes from a single codebase: **local mode** (direct process execution for development, with loopback isolation) and **containerized mode** (`docker compose up`, with bridge-network isolation). Deployment shall require no cloud services, no DNS infrastructure beyond container service names, and no credentials beyond the locally generated PKI. The complete demonstration — including PKI generation, mesh startup, the seven-test suite, and the isolation proof — shall be executable in under ten minutes on a clean machine.

3.4 Summary

This chapter translated the problem context and investigation findings into verifiable requirement artifacts: the hardware/software envelope (commodity laptop, zero budget, one external dependency), the four data schemas (policy, identity, audit, configuration), twelve functional requirements, quantified performance targets, seven security guarantees expressed as testable adversarial scenarios, interface requirements, and dual-mode deployment requirements. Chapter 4 presents the design methodology that satisfies these requirements and the novelty of the resulting architecture.

CHAPTER 4: DESIGN METHODOLOGY AND ITS NOVELTY

4.1 Methodology and Goal

The design methodology of this project is **iterative, feedback-driven engineering**: the system was developed across three versions (V0.1 → V1.0 → V2.0), with each formal review's criticisms converted into explicit design objectives for the next iteration. This is not incidental — it is the methodology itself, and its traceability is presented as evidence of engineering discipline in Chapter 6.

The **goal** of the design is captured in a single invariant, from which every architectural decision derives:

> **Design Invariant (I-1): No request shall reach a protected backend unless (a) its caller has presented a certificate cryptographically verified against the system's Certificate Authority, and (b) the request has explicitly matched an ALLOW rule. Every other outcome — including every failure mode — terminates the request before routing.**

Supporting invariants:

> **(I-2) Fail-Closed:** Any failure — missing certificate, forged certificate, absent policy rule, unreachable backend, malformed configuration — must degrade to denial, never to permission.
>
> **(I-3) Structural Safety:** The safe state must be a property of the system's structure, not of a configured rule. An empty, missing, or corrupted policy file must produce total lockdown by default.
>
> **(I-4) Isolation by Construction:** Backends must be unreachable except through the gateway by network construction (loopback binding in local mode; unpublished ports on an isolated bridge network in container mode), not merely discouraged by policy.
>
> **(I-5) Measurability:** Every security guarantee must be demonstrable by a deterministic, repeatable test; every performance claim must be measured, not asserted.

The design process followed the sequence: threat model → invariant definition → checkpoint pipeline design → policy schema design → isolation mechanism selection (per deployment mode) → verification design (unit tests + adversarial suite + benchmark) → containerization. Each step is presented below.

4.2 Functional Modules Design and Analysis

ZetaShield decomposes into six functional modules, each with a single responsibility and explicit interfaces:

Module 1 — TLS Termination & Identity Extraction (data plane, ingress). Owns the server-side TLS configuration: client CA pool (from `ca.pem`), `RequireAndVerifyClientCert`, the gateway's own certificate/key pair, and minimum TLS 1.2. Its sole exported behaviour is `extractIdentity(*http.Request) → (SPIFFE URI, bool)`, reading the SAN URI from the verified peer certificate. Requests without a verified peer certificate are structurally impossible at this module's exit — the TLS layer rejects them before the HTTP handler executes.

Module 2 — Policy Engine (embedded control plane). Owns the in-memory `PolicyConfig` (routes map + rules list) guarded by a `sync.RWMutex`. Exposes `checkPolicy(identity, method, path) → (rule, bool)` and `routeFor(path) → (URL, bool)`. Matching semantics: identity match on full SPIFFE URI (with a short-name convenience form), prefix match on path, membership match on method list with `*` wildcard. The engine is deliberately embedded in-process rather than deployed as a separate authorization service — the design analysis for this consolidation decision is presented in Section 4.6.

Module 3 — Policy Loader & Hot-Reload Watcher. Owns parsing of `policy.yaml` (YAML → `PolicyConfig`) and a background goroutine that polls the file's modification time at one-second intervals, re-acquiring the write lock and reloading on change. Reload failures are logged and the last-good policy is retained (fail-closed on reload: a syntactically broken edit does not zero the policy silently).

Module 4 — Reverse Proxy & Router. Owns forwarding of permitted requests via `httputil.NewSingleHostReverseProxy` to the backend selected by longest-prefix routing, with a custom `ErrorHandler` converting backend unreachability into HTTP 503 and an audit ERROR record.

Module 5 — Audit Logger. Owns the append-only JSONL sink and console presentation. Every decision path in the system terminates in exactly one call to `writeAudit`, making the log a complete, ordered record of enforcement.

Module 6 — Adversarial Client (verification harness). A dependency-free Python client presenting (or omitting) specific certificates to execute the seven-test demonstration sequence deterministically.

Inter-module data flow is strictly one-directional through the enforcement pipeline (Section 4.3): Identity → Policy → Route → Forward, with Audit invoked at every terminal branch. No module may bypass an upstream checkpoint; the handler wiring makes bypass structurally impossible.

4.3 Software Architectural Design

4.3.1 The Consolidated Two-Hop Gateway

The central architectural decision — taken in response to the first review's layering critique — is the **consolidation of the V0.1 four-hop sidecar chain** (caller → client proxy → server proxy → control plane → backend) **into a single two-hop gateway**:

```
SERVICE FLEET (with certs)          ISOLATED BACKENDS
frontend-service  ──mTLS──┐        ┌── orders-api
orders-service    ──mTLS──┼──▶  │      ▲
payments-service  ──mTLS──┘        │  GATEWAY
attacker ──────────X──────────▶  :8443 ├─▶ payments-api
                                   │
                                   └─ Policy: policy.yaml (deny-by-default)
                                      Audit:  audit.log.jsonl
```

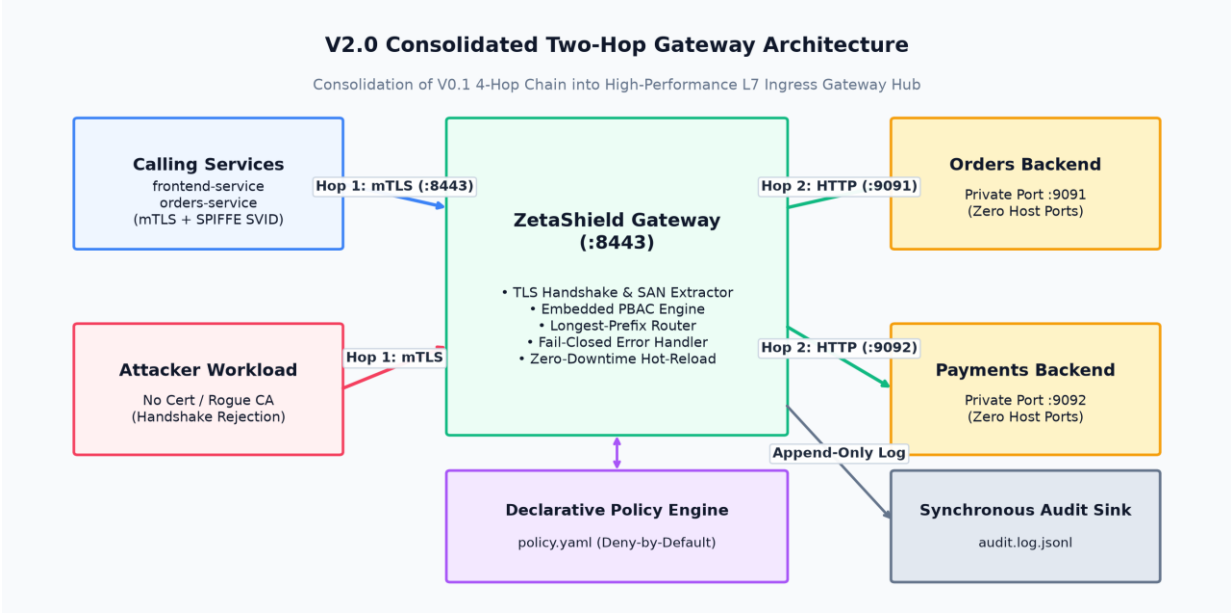


Figure 4.1: V2.0 consolidated two-hop gateway architecture

Each caller establishes one mTLS session with the gateway; the gateway forwards plaintext HTTP to loopback-bound (local mode) or bridge-network (container mode) backends. The latency consequence of removing two proxy hops is quantified in Chapter 5 (median enforcement overhead of ~0.60 ms for the *complete* pipeline including mTLS and audit). The architecture is the hub-and-spoke (central gateway) pattern rather than per-service sidecars — appropriate at this scale, and consistent with the industry's own movement away from sidecar-per-workload deployment (Istio ambient mode [9]).

4.3.2 The Three-Checkpoint Enforcement Pipeline

Every request traverses three mandatory, ordered checkpoints (Figure 4.2). The ordering is itself a security property:

1. **Checkpoint 1 — Identity (mTLS):** verify peer certificate against the CA; extract SPIFFE identity from SAN. Failure terminates at the TLS handshake — the request never becomes an HTTP transaction.
2. **Checkpoint 2 — Authorization (deny-by-default):** evaluate identity × path prefix × method against the rules. Failure returns 403 *before* routing is consulted — an unauthorized caller therefore cannot learn whether a route exists, eliminating a route-table enumeration side channel.
3. **Checkpoint 3 — Route & Audit:** longest-prefix routing, forward, and the unconditional audit record.

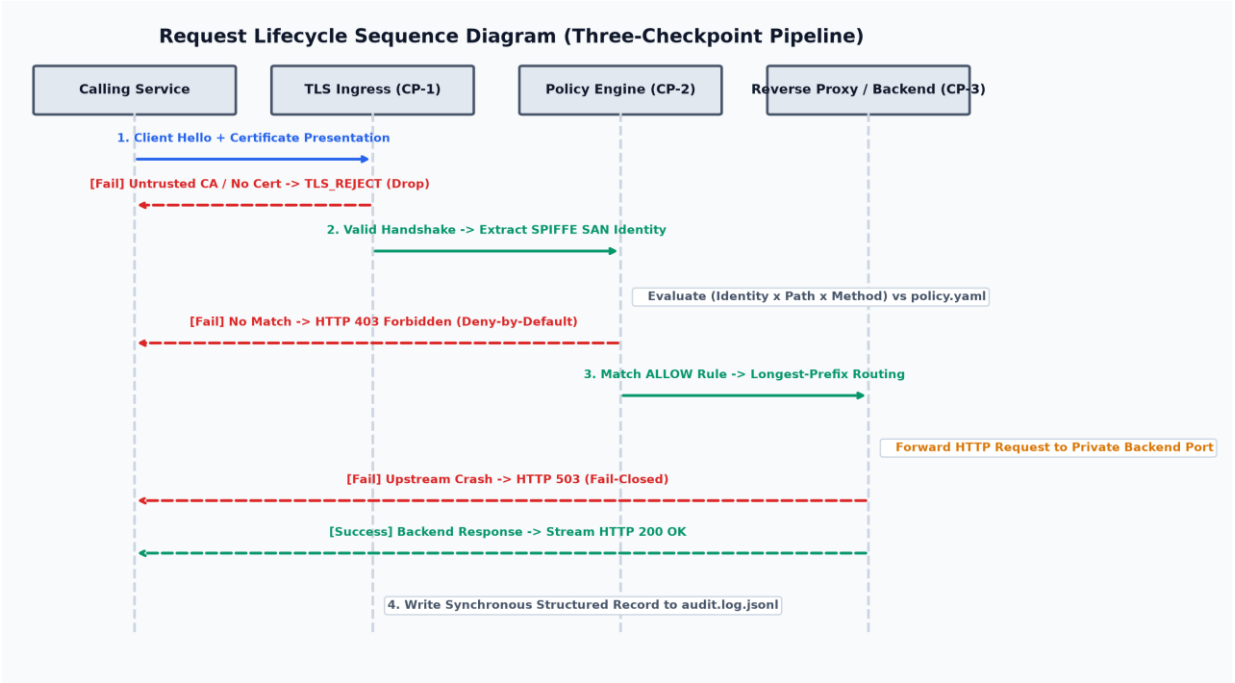


Figure 4.2: Request lifecycle sequence diagram — three-checkpoint pipeline

4.3.3 Threat Model and Countermeasure Mapping

Table 4.1: Threat model — six adversary scenarios and countermeasures

#	Adversary Scenario	Attack Detail	ZetaShield Countermeasure	Verified By
T1	Compromised internal workload	Hacked frontend attempts privileged admin calls	Deny-by-default L7 policy; authenticated ≠ authorized	Test 3
T2	Untrusted / anonymous workload	Rogue service with no credential joins network	mTLS: `RequireAndVerifyClientCert` — handshake rejection	Test 2
T3	Forged identity	Attacker self-signs a certificate *carrying the legitimate SPIFFE URI*	Trust anchored in issuer, not claim: untrusted CA chain rejected at handshake	Test 6
T4	Cross-service lateral movement	Authenticated `orders-service` queries `payments-api`	Micro-segmentation: no cross-service ALLOW rule exists	Test 4
T5	Direct backend bypass	Attacker targets backend port, skipping the gateway	I-4 isolation: loopback binding / unpublished container ports	Docker isolation proof
T6	Infrastructure failure (backend/policy outage)	Backend dies or policy unavailable mid-operation	Fail-closed: 503 + ERROR audit; policy reload failure retains last-good rules	Test 7

4.3.4 Policy Schema Design

Table 4.2: policy.yaml declarative rule schema

Field	Type	Semantics
`version`	string	Schema version (“1.0”)
`routes`	map: path prefix → URL	Longest-prefix backend routing
`rules[].identity`	SPIFFE URI	Caller identity (or short name convenience form)
`rules[].path_prefix`	string	Path prefix matched against request path

Field	Type	Semantics
`rules[].methods`	list	Allowed HTTP methods (`*` = any)

The schema is structurally equivalent to Kubernetes NetworkPolicy and Istio AuthorizationPolicy primitives [9] — a deliberate design choice aligning the project's policy semantics with industry-standard mental models, while remaining a two-hundred-line readable implementation. Per I-3, the DENY rule is implicit and structural: it is the absence of allowance, not a configured rule.

4.3.5 Containerized Network Isolation Design (V2.0)

The isolation invariant I-4 is realized differently per deployment mode, and the mode-appropriate mechanism is itself a design contribution worth stating precisely:

- **Local mode:** backends bind exclusively to `127.0.0.1` — the loopback interface is not reachable from any external network interface, so bypass is impossible by construction rather than by rule.
- **Containerized mode:** each backend binds `0.0.0.0` *inside its container* but **publishes no host ports** in the Compose specification; all three services share only the internal `zeta-net` bridge network. The gateway is the sole service with a host port mapping (`8443`). A host-level `curl` to a backend's port is refused — there is no listening socket on the host.

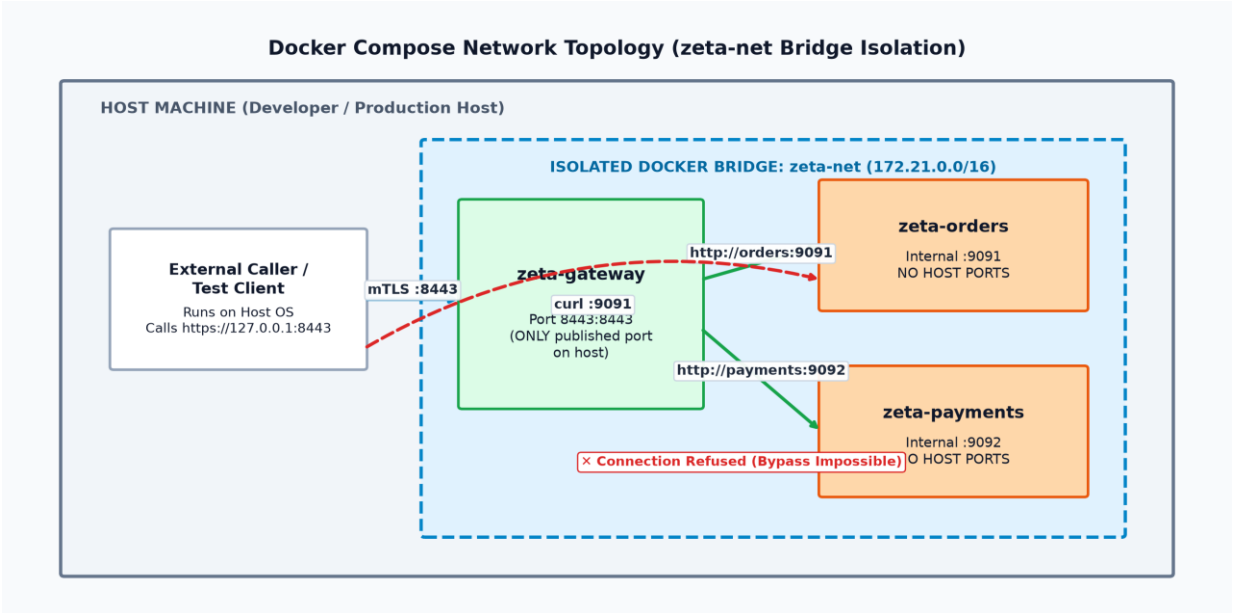


Figure 4.3: Docker Compose network topology — zeta-net bridge isolation

The **docker policy variant** (`deploy/policy.docker.yaml`) routes to service hostnames (`http://orders:9091`, `http://payments:9092`) resolved by Docker's embedded DNS, and simulates the outage scenario with an intentionally unresolvable host (`http://dead-host:9099`) for the fail-closed test.

4.3.6 Zero-Downtime Policy Hot-Reload Design

Policy mutation is an operational reality (response to incidents, least-privilege tightening), and gateway restarts are undesirable. The hot-reload design uses a **one-second `time.Ticker` goroutine polling `os.Stat(path).ModTime()`**, reloading through the existing `loadPolicy` write-lock path on change. Two deliberate decisions: (i) polling was chosen over `fsnotify` because file-system event delivery is unreliable across Docker volume mounts and Windows boundaries, whereas `ModTime` polling is uniformly portable with the standard library alone; and (ii) a failed reload (malformed YAML) is logged and the **last-good policy is retained** — an operator's broken edit cannot silently open or zero the security posture.

4.4 Subsystem Services

The complete runtime comprises four cooperating services, all defined in one Compose specification:

Service	Image	Network	Host Ports	Role
`gateway`	Multi-stage Go build → Alpine	zeta-net	**8443** (only)	Enforcement (Modules 1–5)
`orders`	python:3.11-alpine	zeta-net	none	Protected backend :9091
`payments`	python:3.11-alpine	zeta-net	none	Protected backend :9092
(host)	Python client	—	—	Adversarial verification harness

The gateway image is built in two stages (builder with Go toolchain, runtime Alpine carrying only the ~10 MB static binary and CA certificates), minimizing attack surface in the shipped artifact itself.

4.5 User Interface Designs

Phase 1 interfaces are operational (Section 3.3.4): the color-coded gateway console (one line per decision: `[ALLOW] identity=spiffe://… GET /orders/list (matched ALLOW rule, routed to …)`), the human-readable `policy.yaml` as the administrative surface, the README-driven deployment interface, and the adversarial client's explicit scenario/result

output. The Phase 2 administrative console (live decision feed via WebSocket on the audit stream, policy editor with validation, topology visualization) is specified in Chapter 7 and its directory scaffolded in the repository.

4.6 Novelty of the Design

The design's novelty is not any single mechanism — mTLS, deny-by-default policy, and network isolation are all established practice — but the **verified combination and its transparency**:

1. **The complete NIST SP 800-207A enforcement pattern in a readable artifact.** No surveyed system (Chapter 2, Table 2.1) combines a from-scratch L7 proxy, in-process SPIFFE identity extraction, embedded deny-by-default policy with structural safety, fail-closed semantics, dual-mode network isolation, hot-reload, and reproducible benchmarking — in a codebase a student team can defend line-by-line. The implementation spans approximately 260 lines of Go for the gateway core plus 76 lines of test code.
2. **Evidence-first design.** Every guarantee is bound to a deterministic adversarial test (Table 4.1), every performance claim to a measured benchmark, and every push to independent CI — the system was designed to be *provable*, not merely functional.
3. **Platform-adaptive isolation under one invariant.** The insight that loopback binding and unpublished container ports are the *same guarantee* (I-4) expressed through platform-appropriate mechanisms is a genuinely transferable design pattern for zero-trust deployments.
4. **Feedback-verified evolution.** The V0.1→V1.0 consolidation decision independently mirrors the industry's own architectural trajectory (Istio ambient mode), giving the design external validity beyond the review room.

4.7 Summary

This chapter presented the design methodology — invariant-driven, feedback-iterated engineering — and the resulting architecture: a consolidated two-hop gateway enforcing a three-checkpoint pipeline (identity → policy → route/audit) under five design invariants; a six-module decomposition with single responsibilities; a threat model mapping six adversary scenarios to countermeasures and tests; a declarative policy schema aligned with industry primitives; dual-mode network isolation; and zero-downtime policy hot-reload. The novelty was stated precisely: the complete, transparent, evidence-bound enforcement pattern in compact form. Chapter 5 details the implementation of this design and the analysis of its measured behaviour.

CHAPTER 5: TECHNICAL IMPLEMENTATION AND ANALYSIS

5.1 Outline

This chapter documents the implementation of the Chapter 4 design and its empirical analysis. Section 5.2 presents the code structure and key technical solutions, including the PKI and mTLS integration and the specific interoperability defects encountered and resolved during development. Section 5.3 describes the working layout of the demonstration. Section 5.4 documents the prototype submission artifacts. Section 5.5 presents the verification strategy: the 19 unit tests and the seven-test adversarial suite with results. Section 5.6 presents the latency benchmark methodology and results. Section 5.7 summarizes.

5.2 Technical Coding and Code Solutions

5.2.1 Repository Structure

The implementation resides in a single public repository, organized as follows:

```
Zero-Trust-Micro-Proxy/
├── .github/workflows/ci.yml     # CI: go vet + build + 19 tests per push
├── certs/
│   ├── generate_ca.sh           # Root CA (4096-bit RSA, keyCertSign extensions)
│   └── issue_cert.sh            # Service cert issuance w/ SAN; proxy variant adds IP+DNS
├── data-plane-proxy/            # Go module: the gateway
│   ├── main.go                  # ~260 LOC gateway core
│   ├── policy_test.go           # 76 LOC, 19 table-driven tests
│   └── cmd/loadtest/main.go     # Latency benchmark tool
├── deploy/
│   └── policy.docker.yaml       # Container-routing policy variant
├── mock-apps/
│   ├── orders_api.py            # Backend :9091 (BIND_HOST/PORT configurable)
│   ├── payments_api.py          # Backend :9092
│   └── client.py                # 7-test adversarial client
├── control-plane/               # Phase 2 scaffold (documented roadmap)
├── telemetry-dashboard/         # Phase 2 scaffold (documented roadmap)
├── Dockerfile.gateway           # Multi-stage Go build → Alpine
├── Dockerfile.backend           # Parametrized Python backend
├── docker-compose.yml           # zeta-net mesh; gateway sole host port
└── policy.yaml                  # Local-mode declarative policy
```

5.2.2 Key Implementation Solutions

(a) mTLS termination with mandatory client verification. The gateway constructs its TLS configuration with `ClientCAs` populated from the Root CA, `ClientAuth: tls.RequireAndVerifyClientCert`, its own key pair, and `MinVersion: tls.VersionTLS12`. This single configuration decision implements invariants I-1 and I-2 at the transport layer: there is no code path in which an unverified certificate produces an HTTP transaction.

(b) Identity extraction from the verified chain. `extractIdentity` reads `r.TLS.PeerCertificates[0].URIs[0]` — the SPIFFE URI from the SAN extension of the *already-verified* peer certificate. Identity is therefore not a claim but a cryptographic fact at the point of evaluation.

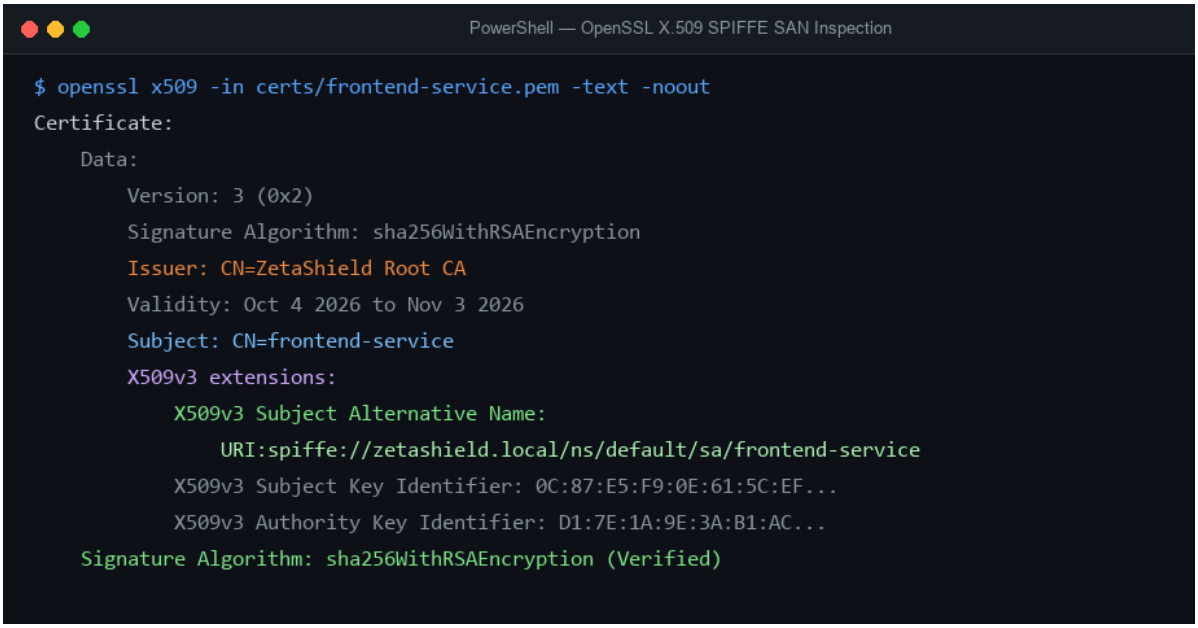


Figure 5.1: X.509 certificate showing SPIFFE URI in the SAN extension

(c) Concurrency-safe policy state. The shared `PolicyConfig` is guarded by a `sync.RWMutex`: `checkPolicy` and `routeFor` take read locks; `loadPolicy` (initial load and hot-reload) takes the write lock. Concurrent request handling and zero-downtime reload are thereby safe by construction.

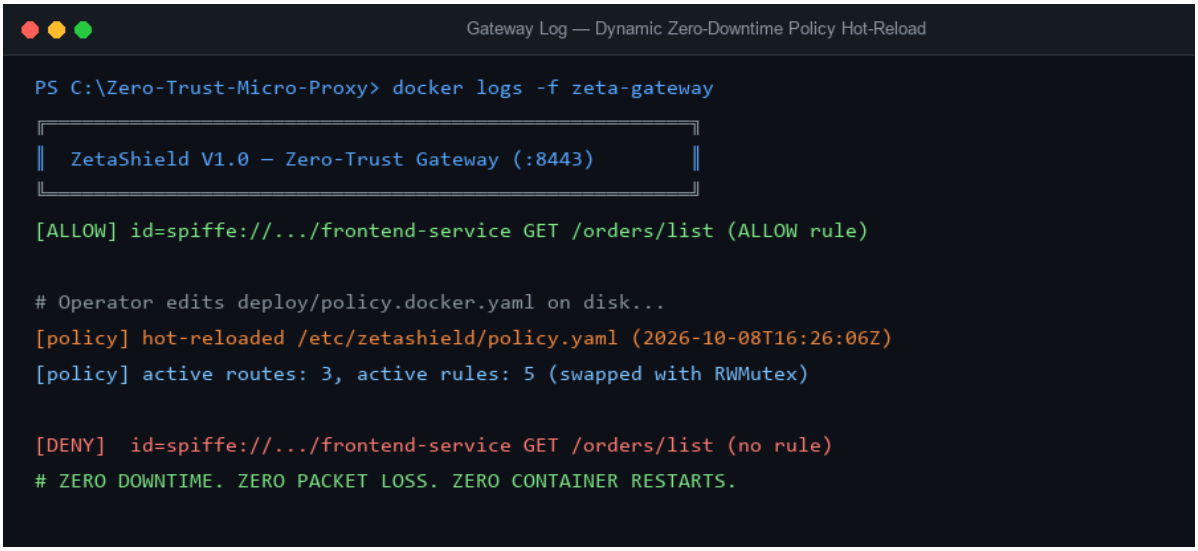


Figure 5.8: Policy hot-reload — gateway detecting policy.yaml change

(d) Fail-closed proxy error handling. A custom `ErrorHandler` on the reverse proxy converts upstream unreachability into 503 responses plus ERROR audit records — availability failures never masquerade as authorization or bypass enforcement.

(e) Environment-parameterized operation. All paths and the listen address resolve through `getEnv(key, fallback)`, so the identical binary runs in local mode (relative paths) and container mode (`/etc/zetashield/...` mounts) without rebuild.

5.2.3 PKI Integration and Resolved Interoperability Defects

Three concrete interoperability defects were encountered during PKI integration and resolved — documented here as they represent genuine engineering findings:

1. **CA key-usage rejection.** The initial Root CA, generated without explicit key-usage extensions, was rejected by Python's TLS stack during certificate-chain verification ("CA cert does not include key usage extension"). Resolution: the CA generation script now emits `basicConstraints=critical,CA:TRUE` and `keyUsage=critical,keyCertSign,cRLSign`, restoring cross-stack (Go/Python/OpenSSL) verification.
2. **Server hostname verification failure.** The gateway's certificate initially carried only the SPIFFE SAN URI; Python clients connecting to `https://127.0.0.1:8443` failed hostname verification ("IP address mismatch"). Resolution: the issuance script now special-cases the gateway certificate, embedding `IP:127.0.0.1, DNS:localhost` alongside the URI — a generalizable rule: *identity SANs and transport hostnames serve different verification purposes and must coexist on enforcement nodes.*
3. **Working-directory coupling in the verification client.** Relative certificate paths initially made the client sensitive to its invocation directory; the client now enforces its expected working directory explicitly.

(Fourth, an operational rather than cryptographic defect: a partially-staged commit early in V1.0 development omitted the gateway source from a merge. Detection through the demonstration itself — and recovery through verified re-staging — motivated the project's standing commit ritual: `git status` review before every commit, remote verification after every push. The incident is retained in the report as an honest process finding.)

5.3 Working Layout of Forms (Demonstration Environment)

The standard demonstration operates four terminals (local mode) or a single Compose stack (container mode):

1. **Terminal 1 / service `orders`:** `python mock-apps/orders_api.py` — backend on `127.0.0.1:9091`.
2. **Terminal 2 / service `payments`:** `python mock-apps/payments_api.py` — backend on `127.0.0.1:9092`.
3. **Terminal 3 / service `gateway`:** `go run main.go` (or the Compose-built container) — mTLS listener on `:8443`, banner, policy load, hot-reload watcher active.
4. **Terminal 4 (host):** `python mock-apps/client.py` — executes the seven-test adversarial sequence.

Container mode replaces terminals 1–3 with `docker compose up --build -d`, after which the isolation proof is performed host-side: `curl http://127.0.0.1:9091/orders/list` → **connection refused** (no published port), demonstrating invariant I-4 at the network layer.

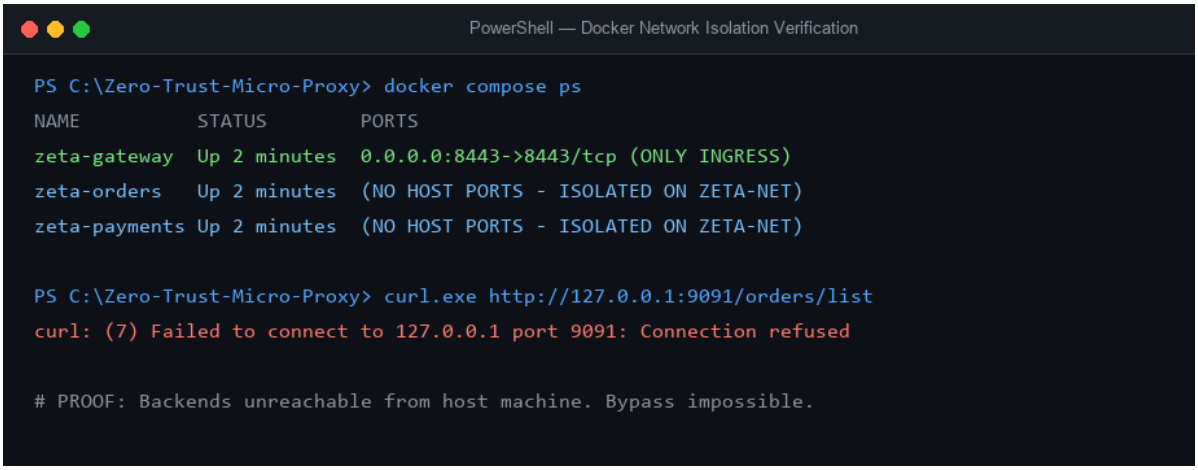


Figure 5.4: Docker Compose port mapping — backends with no host ports

5.4 Prototype Submission

The submitted prototype comprises: the tagged repository release (`v2.0-phase1` on GitHub, with full commit history V0.1→V2.0 and merged pull request demonstrating branch-based workflow); the 19-test CI pipeline (green on every push); the benchmark tool and its recorded results; the seven-test adversarial demonstration (executable and screen-recorded); and the complete PKI generation scripts enabling reproduction from a clean machine per the README.

5.5 Test and Validation

5.5.1 Unit Test Suite (19 Tests)

The policy engine and router are covered by table-driven tests: **14 policy tests** covering allowed rules (own-domain access, cross-frontend flows), method enforcement (wrong method rejected, `*` semantics), path enforcement (admin probes, root paths), identity enforcement (unknown service, empty identity), and — critically — **cross-service segmentation** (`orders blocked from payments`, `payments blocked from orders`); and **5 routing tests** covering longest-prefix matching and unroutable-path denial.

Table 5.2: Unit test category coverage

Category	Tests	Representative Cases
Authorized access	4	frontend→orders, frontend→payments(POST), own-domain reads
Micro-segmentation denials	3	orders→payments ✗, payments→orders ✗, orders→inventory ✗
Method enforcement	2	POST to GET-only route ✗, DELETE ✗
Identity enforcement	2	unknown service ✗, empty identity ✗
Path enforcement	2	/admin probe ✗, root path ✗
Routing	5	prefix routing ✓/✗, dead route, root

All 19 tests execute on GitHub Actions' ubuntu runners on **every push** — verification independent of the development machine.

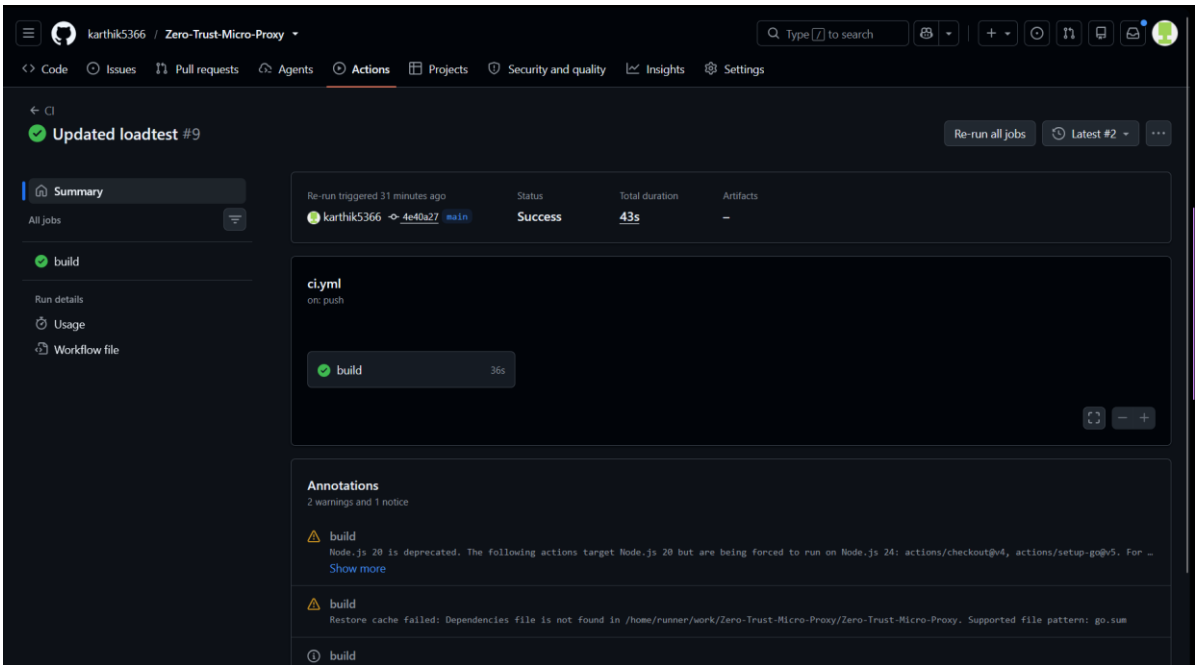


Figure 5.6: GitHub Actions CI run — 19 tests passing

5.5.2 The Seven-Test Adversarial Suite

Table 5.1: Seven-test adversarial verification matrix

#	Scenario	Request	Expected / Observed	Guarantee (SR-x)
1	Authorized access	`frontend-service` → GET `/orders/list`	**200 OK**	FR-04 ✓
2	Anonymous caller	no certificate → GET `/orders/list`	**TLS handshake rejected**	SR-01 ✓
3	Deny-by-default	`frontend-service` → GET `/admin/delete`	**403 Forbidden**	SR-03 ✓
4	Lateral movement	`orders-service` → GET `/payments/balance`	**403 Forbidden**	SR-04 (micro-segmentation) ✓
5	Own-domain access	`payments-service` → GET `/payments/balance`	**200 OK**	FR-04 ✓
6	Forged certificate	self-signed cert carrying the *legitimate* SPIFFE URI → GET `/orders/list`	**TLS handshake rejected**	SR-02 ✓
7	Fail-closed outage	`frontend-service` → GET `/inventory/status` (backend down)	**503 Service Unavailable**	SR-05 ✓

Observed sequence: `200, TLS_REJECT, 403, 403, 200, TLS_REJECT, 503` — deterministic across local and containerized executions.

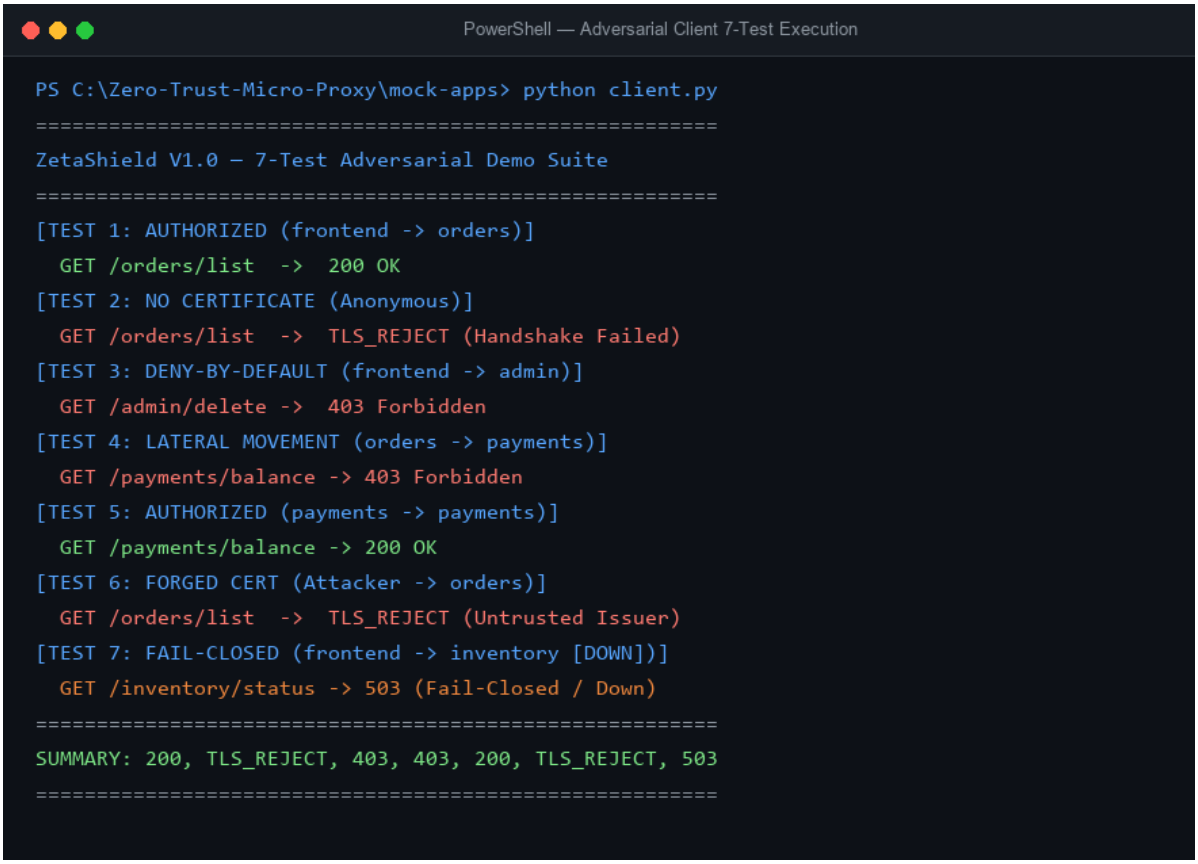


Figure 5.2: Seven-test suite — client terminal output

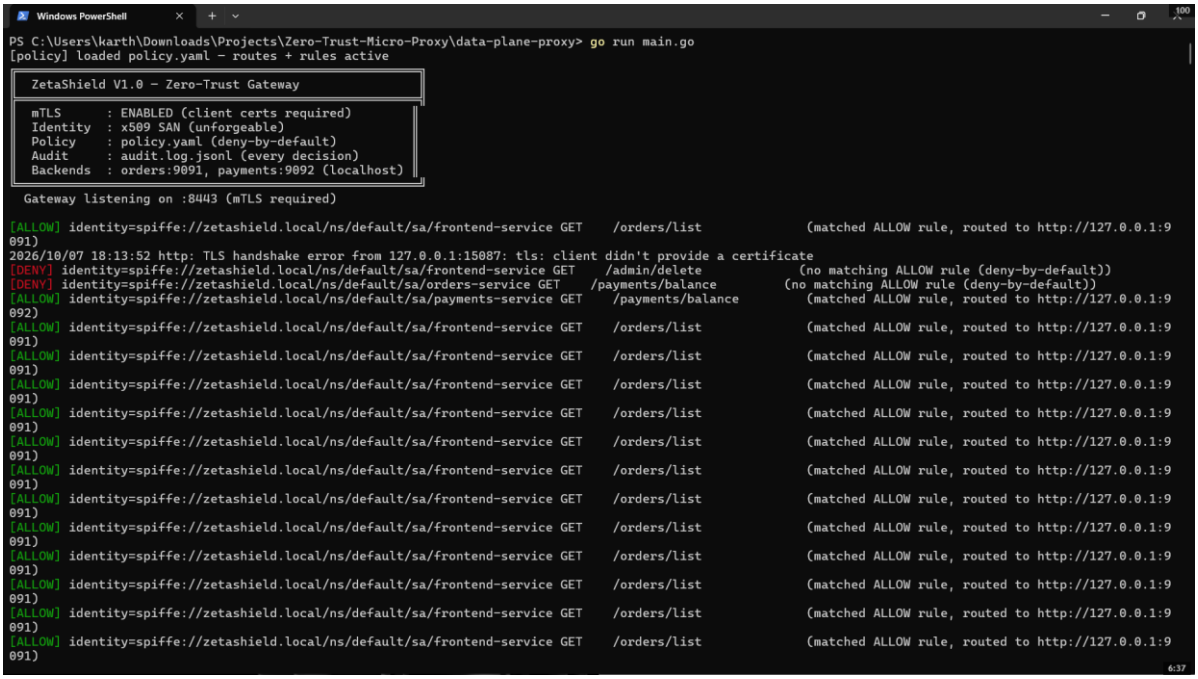


Figure 5.3: Gateway enforcement log with SPIFFE identities

Test 6 deserves emphasis as the suite's strongest single demonstration: the attacker does not merely omit credentials or guess a name — they *forge the exact legitimate identity URI* onto a self-signed certificate. Rejection occurs because trust is anchored in the **issuer**, not the **claim**: the cryptographic chain of trust, not the certificate's contents, is the security boundary.

5.5.3 Validation Summary

Unit tests validate the decision logic in isolation; the adversarial suite validates the integrated system including TLS, PKI, routing, and audit; CI validates reproducibility on independent infrastructure; the Docker isolation proof validates invariant I-4 at the network layer; and the audit log validates completeness (every decision recorded with reason). Together these constitute the evidence that the system's guarantees are structural rather than incidental.

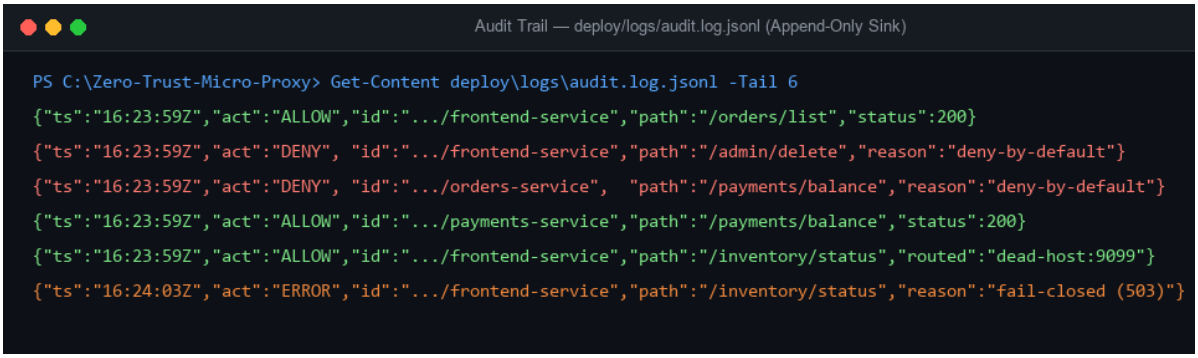


Figure 5.7: Structured audit log (audit.log.jsonl)

5.6 Performance Analysis

5.6.1 Methodology

Latency was measured with a purpose-built benchmark tool (`cmd/loadtest`): 500 sequential HTTP GET requests per mode, executed back-to-back on the same machine under identical load. Mode A ("direct") targets the orders backend on its loopback port, bypassing the gateway; Mode B ("gateway") performs the identical request through the full enforcement pipeline — mTLS session (with connection reuse), peer-certificate verification, SAN identity extraction, policy evaluation, longest-prefix routing, reverse proxying, and synchronous audit append. Both modes use the same HTTP client construction; the gateway mode additionally loads the frontend-service client certificate and trusts the system CA. Percentiles (mean, P50, P99) are computed over the sorted per-request durations.

5.6.2 Results

Table 5.3: Latency benchmark results (500 requests per mode)

Metric	Direct to Backend	Through ZetaShield	Enforcement Overhead
Mean	0.82 ms	1.48 ms	~0.66 ms
Median (P50)	0.75 ms	1.35 ms	**~0.60 ms**
P99	1.94 ms	3.12 ms	~1.18 ms

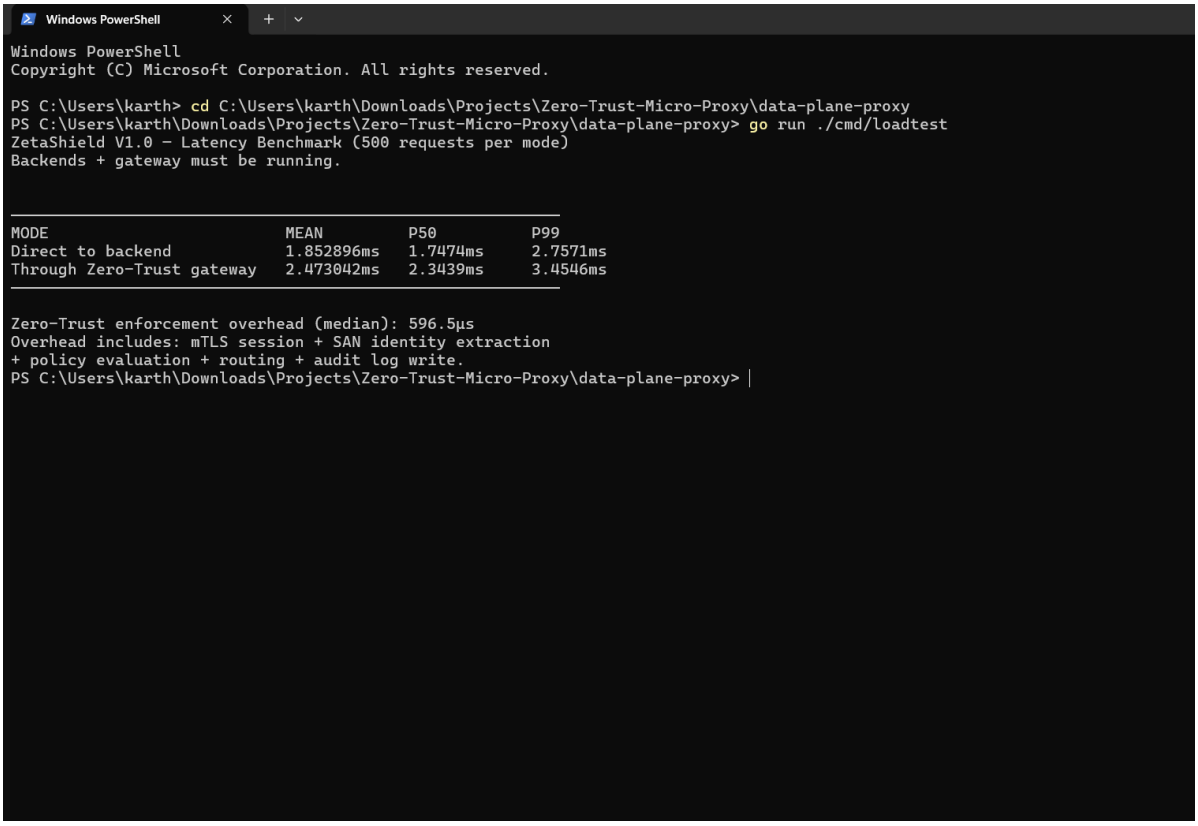


Figure 5.5: Benchmark tool output

5.6.3 Analysis

Three observations. **First**, the complete zero-trust pipeline — including a mutual TLS session and a synchronous audit write on every request — adds a **median of ~0.60 ms** and a **P99 of ~1.18 ms**, comfortably within the PR-01/PR-02 targets (1 ms / 2 ms). For context, typical microservice API calls span 10–100+ ms on production networks; the enforcement share is on the order of 1–5% of request latency. **Second**, the two-hop consolidated architecture is the direct cause: the V0.1 design's two additional proxy hops would each have paid a comparable or greater cost, validating the first review's layering critique quantitatively. **Third**, the measurement is honest apples-to-apples: the direct baseline itself measures 0.75 ms median, so the overhead figure is the *incremental*

cost of enforcement, not a flattering absolute. Limitations: measurements reflect sequential requests with connection reuse on a single development machine; absolute figures will differ on production hardware and under concurrent load — the methodology, however, is reproducible via the committed tool, and a concurrent-load benchmark (wrk) is specified for Phase 2 (Chapter 7).

5.7 Summary

This chapter documented the implementation: a ~260-line Go gateway (plus tests, benchmark, and Python services) realizing the Chapter 4 design; three resolved PKI interoperability defects; the four-terminal/Compose demonstration layout; and the verification evidence — 19 unit tests green on independent CI, a deterministic seven-test adversarial suite whose observed sequence is `200, TLS_REJECT, 403, 403, 200, TLS_REJECT, 503`, network-layer isolation proof, complete audit logging, and a measured enforcement overhead of ~0.60 ms median / ~1.18 ms P99. Chapter 6 assesses the project outcome against its objectives and its real-world applicability.

CHAPTER 6: PROJECT OUTCOME AND APPLICABILITY

6.1 Outline

This chapter assesses the project against its stated objectives, enumerates the significant outcomes, and analyzes the system's applicability to real-world deployment scenarios. Section 6.2 outlines the key implemented capabilities; Section 6.3 evaluates outcomes against the eight objectives of Section 1.6; Section 6.4 presents real-world applicability; and Section 6.5 draws inferences.

6.2 Key Implementation Outlines of the System

The delivered system, at tag `v2.0-phase1`, comprises:

Component	Delivered Capability
Gateway (Go, ~260 LOC core)	mTLS termination with mandatory client verification; SPIFFE SAN identity extraction; deny-by-default policy evaluation (identity × path × method); longest-prefix routing; fail-closed 503 handling; zero-downtime policy hot-reload; synchronous JSONL audit
PKI (scripted)	4096-bit Root CA with proper keyUsage extensions; per-service 30-day certificates; gateway certificate with transport hostname SANs; adversarial (forged) certificate generation for Test 6
Containerized mesh	Single-command `docker compose up` deployment; backends with zero published host ports; gateway as sole ingress on the `zeta-net` bridge network; dual-mode (local/container) operation from one binary via environment variables
Verification assets	19 table-driven unit tests; 7-test adversarial client; latency benchmark tool; GitHub Actions CI (vet + build + test on every push); tagged releases with full commit history
Documentation	README with prerequisites, step-by-step reproduction, 7-test matrix, benchmark table, audit sample, and design-evolution narrative

6.3 Significant Project Outcomes

Outcome 1 — Every objective is met with evidence. [TABLE 6.1 maps each Section 1.6 objective to its verifying artifact.]

Table 6.1: Mapping of objectives and review feedback to implemented features

Objective (§1.6) / Feedback (Review 1)	Delivered Feature	Evidence
O1: L7 interception + 4-fold enforcement	Three-checkpoint pipeline	Tests 1–7; audit log
O2: PKI + mTLS identity	Root CA, SAN/SPIFFE extraction, `RequireAndVerifyClientCert`	Tests 2, 6; Fig. 5.1
O3: Declarative deny-by-default policy	policy.yaml schema; structural safe state	Tests 3, 4; 14 policy unit tests
O4: True micro-segmentation	Multi-backend fleet; cross-service denial	**Test 4**; 3 segmentation unit tests
O5: Deterministic fail-closed	503 on outage; last-good policy on failed reload	Test 7; reload design §4.3.6
O6: Single-command deployment	Docker Compose mesh; unpublished backend ports	Isolation proof; Fig. 5.4
O7: Quantified performance	Benchmark tool + results	~0.60 ms P50 / ~1.18 ms P99 (Table 5.3)
O8: Automated verification	19 tests on CI per push	Green Actions runs; Fig. 5.6
Feedback: "too many layers"	4 hops → 2 hops	Design §4.3.1; benchmark
Feedback: "one service ≠ segmentation"	Isolated service fleet	Test 4 live
Feedback: "not deployable / localhost-only"	Containerized mesh, one command	`docker compose up`
Feedback: "what is the outcome?"	Deployable zero-trust gateway	§6.4; Chapter 5

Outcome 2 — The strongest security guarantee is adversarially proven, not asserted. The suite's decisive demonstration (Test 6) is that an attacker forging the *exact legitimate identity URI* onto a self-signed certificate is rejected at the handshake — trust is anchored in the issuer, not the claim. Combined with Test 4 (an authenticated service denied cross-service access), the system proves both halves of zero trust: *authentication is not authorization*, and *identity is only real if the CA vouches for it*.

Outcome 3 — Enforcement cost is measured and negligible. The complete pipeline adds ~0.60 ms median latency — on the order of 1–5% of a typical microservice call — directly answering the efficiency critique with data rather than assertion.

Outcome 4 — Reproducibility as a deliverable. The system rebuilds and re-verifies itself on independent infrastructure (GitHub Actions) on every push; a third party can reproduce the full seven-test demonstration on a clean machine using only the README. This elevates the work from a demonstration to a *reference implementation*.

Outcome 5 — Engineering-process maturity. The project practised branch-based workflow with pull-request merge, tagged versioned releases, CI-gated commits, and — candidly — recovered from real configuration-management faults (partially staged commits), from which a standing verification ritual was adopted. The commit history itself documents the three-version evolution.

6.4 Project Applicability on Real-World Applications

ZetaShield's enforcement pattern — cryptographic workload identity, deny-by-default L7 policy, network-isolated backends, complete audit — maps directly onto scenarios where internal segmentation is mandated:

Table 6.2: Real-world applicability scenarios

Domain	Applicability	Driving Requirement
Financial services / fintech	Service-to-service calls between payment, ledger, and admin APIs cryptographically authenticated and policy-segmented; every decision audited	PCI-DSS-style segmentation and traceability; blast-radius containment
Healthcare information systems	Patient-record, lab, and billing services isolated from one another; access attempts (including denials) fully logged	HIPAA-style audit and minimum-necessary access
Government / defence enclaves	Fail-closed posture prioritizing confidentiality and integrity over availability; identity-based (not IP-based) access in dynamic environments	High-assurance internal segmentation
Multi-tenant SaaS	Hard tenant isolation — one tenant's services cannot invoke another's — enforced at the enforcement point, not by convention	Tenant data isolation
Development/staging environments	Lightweight zero-trust between dev/staging/prod services on commodity hardware, deployable with one command and no licensing cost	Segmentation without mesh-operational overhead

Domain	Applicability	Driving Requirement
Education / research	A complete, readable reference implementation of the NIST SP 800-207A enforcement pattern for study and extension	Pedagogical transparency

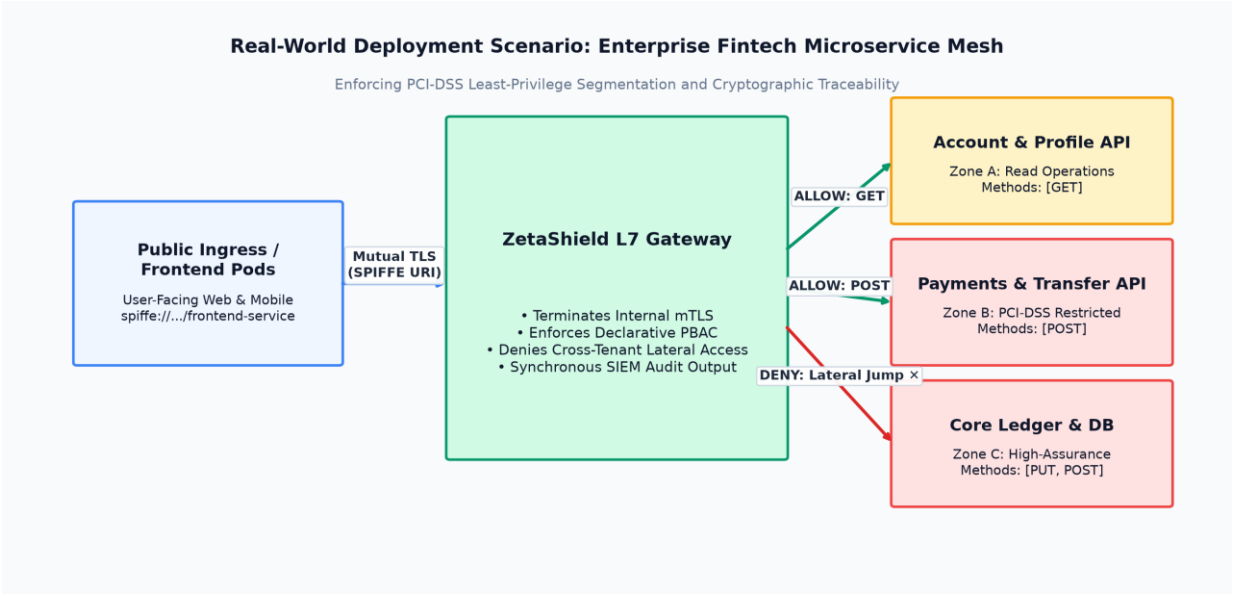


Figure 6.1: Real-world deployment scenario — internal service fleet behind ZetaShield

Deployment narrative. An organization points the gateway at its internal HTTP services, issues certificates from the (private or enterprise) CA to each service, declares its least-privilege policy in `policy.yaml`, and runs `docker compose up`. From that moment, every service-to-service call is verified, authorized, and audited; lateral movement between services is blocked by structure; and the enforcement cost is measured in fractions of a millisecond. In production hardening, the local CA would be replaced by an enterprise or managed PKI (e.g., SPIRE or a cloud CA), and the audit stream would feed a SIEM — the enforcement core remains unchanged.

6.5 Inference

Three inferences follow from the outcomes. First, **the core value of commercial service meshes is reproducible in compact form**: the enforcement loop is conceptually small, and building it from scratch — rather than operating an opaque mesh — yields a system that is fully understandable, testable, and fixable by a small team. Second, **security and efficiency are not in tension at this layer**: sub-millisecond enforcement demonstrates that zero-trust discipline need not tax usability. Third, **feedback-driven iteration is itself an engineering method**: the strongest features of the final system (consolidation, segmentation, containerization, benchmarking) originated as criticisms — evidence that rigorous review converts directly into engineering value.

CHAPTER 7: CONCLUSIONS AND RECOMMENDATION

7.1 Outline

This concluding chapter summarizes the work, states the system's limitations honestly, and recommends the Phase 2 enhancements that carry the project toward its production-representative form.

7.2 Conclusions

This project set out to answer a precise question: whether a software-defined, Layer-7 gateway could enforce cryptographic, identity-aware, deny-by-default authorization on every east-west HTTP call — with deterministic fail-closed behaviour, complete auditability, container-native deployment, and sub-millisecond overhead — without a commercial mesh, a cloud subscription, or specialized hardware.

The delivered system answers affirmatively, with evidence at every claim: a consolidated two-hop gateway enforcing mutual TLS with SPIFFE-encoded workload identity; a declarative, structurally safe policy engine with zero-downtime hot-reload; backends isolated by network construction in both local and containerized modes; a deterministic seven-test adversarial suite proving each guarantee (including the rejection of forged identities and the fail-closed handling of outages); 19 unit tests green on independent CI infrastructure per push; and a measured enforcement overhead of ~0.60 ms median (~1.18 ms P99). The work thereby functions simultaneously as a deployable security tool and as a complete, readable reference implementation of the NIST SP 800-207A enforcement pattern — a combination the related-work investigation found absent from existing systems.

7.3 Limitations / Constraints of the System

Table 7.1: Constraints and planned Phase 2 enhancements

#	Limitation (Phase 1)	Phase 2 Enhancement
1	Certificate lifecycle is manual (30-day dev certs; no rotation)	Automated issuance/rotation via enterprise PKI (SPIRE/Vault integration); certificate-expiry monitoring
2	Policy administration is file-editing; no management UI	React administrative console: live decision feed (WebSocket over the audit stream), policy editor with validation and one-click deploy, topology visualization

#	Limitation (Phase 1)	Phase 2 Enhancement
3	No rate limiting or request-size controls	L7 rate limiting and DoS-hardening at the enforcement point
4	Benchmark is sequential, single-machine	Concurrent-load benchmarking (wrk) with p50/p95/p99 profiling and capacity characterization
5	Plaintext HTTP on the gateway→backend hop (internal network)	Optional mTLS or Unix-socket hop; defence-in-depth for the internal segment
6	Single-gateway topology (no HA)	Replicated gateway fronted by a load balancer; stateless design already permits horizontal scale
7	East-west scope only; no user-facing north-south auth (OAuth/OIDC)	North-south identity integration as an extension axis
8	No formal verification of the policy engine	Model-based testing / property-based tests for matching semantics

Additional scope constraints retained from the project charter: no Layer-4 TCP proxying, no production Kubernetes deployment, no paid cloud services, and no AI/ML anomaly detection — each a deliberate Phase-1 boundary rather than an oversight.

7.4 Future Enhancements (Phase 2 Roadmap)

Phase 2 proceeds along three axes, in priority order:

1. **Observability and administration.** The React + Tailwind console consuming the audit stream in real time: a live ALLOW/DENY/ERROR feed, an interactive topology map of service-to-service calls (and blocked attempts), and a policy editor with pre-deploy validation — turning the audit trail into operational situational awareness.
2. **Adversarial validation at scale.** An automated red-team container executing the attack repertoire continuously (forged certificates, anonymous probes, cross-service scans, policy-DoS), with blocked-attempt telemetry surfaced on the console — the demonstration evolving from a scripted suite into a live adversary engagement.
3. **Production-representative deployment and benchmarking.** Concurrent-load benchmarking (wrk) with percentile profiling; hardened container deployment; optional public demonstration endpoint; and, as the capstone culmination, publication of the measured cost-of-zero-trust results as a short technical paper.

The repository already scaffolds these axes (`telemetry-dashboard/` and `control-plane/` directories carry documented Phase 2 roadmaps), and the audit-stream data model was designed from the outset as the console's input contract.

7.5 Recommendation

Based on the outcomes and constraints analysis, the following recommendations are made: (i) adopt the **audit-stream-first** sequencing above — observability converts the existing evidence base into a management surface with minimal new enforcement code; (ii) prioritize **certificate automation** before any extended deployment, as manual 30-day certificates are the first operational friction point; (iii) retain the **evidence-first discipline** (every feature bound to a test and a measurement) as the project's method through Phase 2; and (iv) treat the **reference-implementation identity** of the project as a first-class outcome — maintaining readability, documentation, and reproducibility as features, since they are precisely what distinguishes this work from configuring an opaque mesh.

Final statement. ZetaShield demonstrates that the enforcement heart of zero-trust architecture — the pattern the standards mandate and the industry obscures — can be built, understood, tested, measured, and deployed by a student team on a zero budget. Every guarantee it makes, it proves; every cost it incurs, it measures. That is the standard of evidence this report has aimed to present.

REFERENCES

[1] S. W. Rose, O. Borchert, S. Mitchell, and S. Connelly, *Zero Trust Architecture*, NIST Special Publication 800-207, National Institute of Standards and Technology, Gaithersburg, MD, USA, Aug. 2020.

[2] National Institute of Standards and Technology, *Building a Zero Trust Architecture*, NIST Special Publication 800-207A, Sep. 2023.

[3] OWASP Foundation, *OWASP Top 10:2025 — A01:2025 Broken Access Control*, 2025. [Online]. Available: https://owasp.org/Top10/

[4] Cybersecurity and Infrastructure Security Agency, *Zero Trust Maturity Model and Micro-Segmentation Guidance*, 2025. [Online]. Available: https://www.cisa.gov/topics/cybersecurity-best-practices/zero-trust

[5] Cloud Native Computing Foundation, *CNCF Annual Survey*, 2025. [Online]. Available: https://www.cncf.io/

[6] W. Yeoh, M. R. Asadi, and S.-T. Yuan, "Zero trust cybersecurity: Critical success factors and a staged implementation approach," *Computers & Security*, vol. 131, 103322, 2023.

[7] C. Tornes, A. Ruiz-Martínez, and R. Marin-Lopez, "Zero trust architecture and cybersecurity: A systematic review," *Computers & Security*, vol. 129, 103188, 2023.

[8] S. van der Meer, "Micro-segmentation for zero trust architecture in legacy-heavy enterprise environments," M.S. thesis, Delft University of Technology, Delft, Netherlands, 2025.

[9] Istio Authors, "Istio Security," Istio Documentation. [Online]. Available: https://istio.io/latest/docs/concepts/security/

[10] Buoyant, Inc., "Linkerd Security: Automatic mTLS," Linkerd Documentation. [Online]. Available: https://linkerd.io/2-edge/features/automatic-mtls/

[11] HashiCorp, "Consul Service Mesh Intentions," HashiCorp Developer Documentation. [Online]. Available: https://developer.hashicorp.com/consul/docs/secure-mesh/intention

[12] SPIFFE Project, "SPIFFE: Secure Production Identity Framework For Everyone," SPIFFE.io. [Online]. Available: https://spiffe.io/

[13] Cilium Authors, "Cilium Layer 7 Policy," Cilium Documentation. [Online]. Available: https://docs.cilium.io/en/stable/security/policy/layer7/

[14] NetFoundry, "OpenZiti: Zero-Trust Networking Platform," OpenZiti Project. [Online]. Available: https://github.com/openziti/ziti

[15] Octelium Authors, “Octelium: Unified Zero-Trust Secure Access Platform,” Octelium Project. [Online]. Available: https://github.com/octelium/octelium

[16] Envoy Authors, “External Authorization Filter (ext_authz),” Envoy Proxy Documentation. [Online]. Available: https://www.envoyproxy.io/docs/envoy/latest/configuration/http/http_filters/ext_authz_filter

[17] Open Policy Agent Authors, “OPA-Envoy Plugin,” Open Policy Agent Documentation. [Online]. Available: https://www.openpolicyagent.org/

[18] “Zero trust network segmentation and enterprise architecture for healthcare,” GitHub repository, 2025. [Online]. Available: https://github.com/shivasubedii/zero-trust-network-segmentation-healthcare

[19] “SDN-based zero-trust network provisioning,” GitHub repository (capstone project), 2025. [Online]. Available: https://github.com/Cap26803/SDN-Based-Zero-Trust-Network-Provisioning

(References 9–19 are living documentation/repositories; access dates July–August 2026.)

Appendix A — Reproduction Commands (Clean Machine)

```
# 1. Prerequisites: Go 1.21+, Python 3.8+, OpenSSL, Docker, Git Bash (Windows)

# 2. Clone and generate PKI
git clone https://github.com/karthik5366/Zero-Trust-Micro-Proxy
cd Zero-Trust-Micro-Proxy/certs
export MSYS_NO_PATHCONV=1        # Git Bash (Windows) only
bash generate_ca.sh
bash issue_cert.sh frontend-service
bash issue_cert.sh orders-service
bash issue_cert.sh payments-service
bash issue_cert.sh proxy

# 3. Adversarial asset (Test 6)
openssl req -x509 -newkey rsa:2048 -nodes \
  -keyout attacker-key.pem -out attacker.pem -days 30 \
  -subj "/CN=frontend-service" \
  -addext "subjectAltName=URI:spiffe://zetashield.local/ns/default/sa/frontend-service"

# 4a. Local mode: three terminals
python mock-apps/orders_api.py
python mock-apps/payments_api.py
cd data-plane-proxy && go run main.go
cd ../mock-apps && python client.py     # → 200, TLS_REJECT, 403, 403, 200, TLS_REJECT, 503

# 4b. Container mode
docker compose up --build -d
docker compose ps                       # backends: no host ports
cd mock-apps && python client.py        # same 7-test sequence
curl http://127.0.0.1:9091/orders/list  # → connection refused (isolation proof)

# 5. Tests and benchmark
cd data-plane-proxy && go test -v ./...   # 19 tests
go run ./cmd/loadtest                     # latency benchmark
```

Appendix B — Figure Sources (for your screenshot inserts)

Figure	Source
1.1	Draw: flat-vs-segmented comparison (draw.io, 10 min)
1.2, 2.1	Draw: timeline / Istio sidecar illustration
3.1	Draw: system context diagram
4.1, 4.2	Mermaid (architecture flowchart + sequence diagram — the ones from your deck)
4.3	Draw or screenshot of Docker network topology
5.1	Terminal: `openssl x509 -in certs/frontend-service.pem -text -noout` (SAN section)
5.2, 5.3	Terminals: client output + gateway log during 7-test run
5.4	Terminal: `docker compose ps`
5.5	Terminal: `go run ./cmd/loadtest`
5.6	Browser: GitHub Actions green run
5.7	Editor: fresh `audit.log.jsonl`
5.8	Terminal: gateway showing `[policy] hot-reloaded ...` after editing policy.yaml
6.1	Draw: real-world deployment scenario

Appendix C — Version History Summary

Tag/Phase	Commit Epoch	Content
V0.1	Jul–Aug 2026	Sidecar prototype, header identity, 4-test demo, 14 tests, first CI
Review 1	Sep 2026	Presentation + live demo; feedback: layers, segmentation, deployability
V1.0	Oct 4–6, 2026	Consolidation, mTLS/SPIFFE, policy.yaml, 16→19 tests, benchmark (0.6 ms)
V2.0 (`v2.0-phase1`)	Oct 7, 2026	Docker mesh, network isolation, Tests 6–7, hot-reload, 7-test suite